

第十九届“挑战杯”全国大学生课外学术科技作品 竞赛“揭榜挂帅”专项赛

项目设计报告

选题名称

SH-10 无人机双机协同自主搜救任务

作品名称

面向自主搜救任务的双机系统智能决策算法研究

学校

北京航空航天大学

成员

郑宇杰、传真奇、张家文、杨佳宇轩

目录

目录

- 项目简介
- 项目背景
- 项目目标
- 项目实施
 - 阶段一：任务启动与广域巡航
 - 阶段二：目标搜索、识别与信息共享
 - 阶段三：精准抵近与末端作业
 - 阶段四：任务完成与返航
- 项目结构
 - 图示
 - 部分介绍

	src/offboard_run
6. 开发过程	
6.1 整体流程	
6.2 项目分工	
7. 文件说明	
7.1 vtol_commander.py（阶段一）	
7.1.1 整体路线预览	
7.1.2 起飞降落逻辑	
7.1.3 避障策略	
7.2 detect_stage1.py（阶段二）	
7.3 iris_commander.py（阶段三）	
7.3.1 整体路线预览	
7.3.2 控制逻辑	
7.3.3 避障策略	
7.4 detect_white.py & detect_red.py（阶段四）	
8. 关键代码及注释	
8.1 detect_stage1.py代码及注释	
8.2 vtol_commander.py代码及注释	
8.3 iris_commander.py代码及注释	
8.4 detect_red.py 代码及注释	
8.5 detect_white.py 代码及注释	
9. 运行须知	
9.1 启动方式	
9.2 依赖项	
1. OpenCV	
2. YOLOv8	
3. cv_bridge	
4. gazebo_msgs	

1. 项目简介

本项目聚焦于“无人机双机协同自主搜救”这一前沿挑战，设计并实现了一套面向复杂动态环境的异构无人机协同智能决策系统。系统由一架垂起固定翼无人机（VTOL）作为广域侦察单元和一架四旋翼无人机作为精准响应单元构成。

任务流程分为两个核心阶段：首先，VTOL无人机利用其航程远、效率高的优势，在大范围内执行自主航路规划、动态模式切换、躲避禁飞区及多目标实时感知任务，通过搭载的YOLOv8视觉感知模块，实现对三类模拟人员的精确识别与世界坐标解算。在完成侦察并回传目标数据后，四旋翼无人机接替任务，基于接收到的目标信息执行高精度路径规划与末端视觉伺服控制，分别对静态及动态目标进行稳定追踪与抵近，最终在要求的高度上，完成关键信息的精确发布，全过程无需人工干预。

2. 项目背景

无人机技术在环境监测、应急通信和抢险救灾等领域的应用正迅速深化。尤其在灾后搜救等复杂场景中，地面人员往往面临视野受限、地形阻碍及二次坍塌等重大安全风险。无人机凭借其空中优势，能够快速获取全局态势，显著提升搜救效率并降低人员风险。

然而，单一无人机平台难以兼顾大范围搜索效率与近距离精准操作的需求。为此，**异构双机协同**已成为解决此类复杂任务的前沿范式。该模式利用VTOL无人机兼具多旋翼垂直起降灵活性与固定翼长航时巡航能力的特点，执行广域快速扫描；同时利用四旋翼无人机优越的悬停与机动性能，执行抵近侦察与精准任务操作。本项目正是在此背景下，针对包含禁飞区、多类别动态/静态目标识别以及双机任务交接的复杂协同搜救场景，研发一套高效、鲁棒的智能决策与控制算法，旨在验证并提升异构无人机集群在该类任务中的自主协同作业能力。

3. 项目目标

本项目的核心目标是设计、实现并验证一套应用于异构双机协同搜救任务的**智能决策与控制算法**。该系统旨在模拟真实搜救场景，验证无人机在GPS信号存在噪声、目标部分动态的挑战下，从起飞到任务完成的全流程自主作业能力。

具体子目标分解如下：

- 高层任务规划与导航**：实现 VTOL 无人机的全自主起降、飞行模式无缝切换，以及融合了禁飞区的全局路径规划与航点跟踪能力。
- 实时环境感知与目标定位**：开发基于 YOLOv8 的机载视觉感知模块，实现对多类别目标的实时检测、分类，并构建从像素坐标到全局地理坐标的精确解算模型。
- 动态目标追踪与运动预测**：针对动态目标，设计一套轨迹数据采样与运动模型拟合算法，实现对目标未来位置的有效预测，为精准拦截提供决策依据。
- 末端精准伺服与信息发布**：为四旋翼无人机开发基于视觉反馈的闭环伺服控制算法，实现对目标的稳定抵近、对心悬停，并在满足竞赛规则的高度与精度要求下，完成关键信息的发布。

4. 项目实施

4.1 阶段一：任务启动与广域巡航

无人机A (`standard_vtol_0`) 执行任务初始化序列。首先以多旋翼模式垂直起飞至预定安全高度，随后执行**多旋翼至固定翼的飞行模式转换**。进入固定翼模式后，激活全局路径规划模块，结合禁飞区约束，生成一条绕开禁飞区的无碰撞航路，最终飞抵指定的GPS引导点。

4.2 阶段二：目标搜索、识别与信息共享

抵达GPS引导点后，无人机A进入搜索模式，开始执行**预设的覆盖式路径规划航线以确保对指定搜索区域的完全覆盖**。在此期间，**机载的视觉感知模块**被激活，该模块基于YOLOv8深度学习模型，对视频流进行实时处理，以检测和分类三类目标（危重、轻伤、健康）。一旦检测到目标，系统将通过坐标解算算法获取其全局Pose，并通过ROS话题向外发布，实现与无人机B的机间数据链模拟传输。完成所有目标的搜索与信息发布后，无人机A自主返航至旋翼模式区，执行固定翼至多旋翼的模式转换，并最终垂直降落、自动锁桨。

4.3 阶段三：精准抵近与末端作业

无人机A安全着陆后，无人机B (`iris_0`) 自动解锁起飞。任务规划模块根据接收到的目标信息进行决策，首先规划前往健康人员（动态目标）的航路。抵达目标区域后，激活**末端视觉引导系统**：

- 对于**健康人员**：进入目标状态估计阶段。通过连续的视觉检测，对目标的运动轨迹进行数据采样与拟合，建立其运动模型。基于该模型，计算出最优拦截点并前出至该点上空，最终下降至0.7米高度等待目标进入指定区域后发布其平台坐标。

- 对于**重伤人员**：激活视觉伺服控制逻辑。采用“对准-下降”的递进策略，通过闭环控制不断修正无人机水平位置，确保其精确位于目标正上方，并逐步下降至0.5米高度发布其平台坐标。

4.4 阶段四：任务完成与返航

无人机B在完成对两个指定目标的任务后，执行返航程序，自主飞行返回旋翼模式区，并执行自动降落、锁桨，标志着整个协同搜救任务的圆满完成。

5. 项目结构

5.1 图示

项目的所有文件位于工作区`catkin_ws/src/offboard_run`下

```
.
├── CMakeLists.txt
├── README.md
├── models
│   ├── red.pt
│   ├── stage1.pt
│   ├── stage2.pt
│   └── white.pt
├── msg
├── package.xml
├── score_check.py
├── scripts
│   ├── __pycache__
│   ├── detect_red.py
│   ├── detect_stage1.py
│   ├── detect_white.py
│   ├── iris_commander.py
│   ├── launch.sh
│   └── vtol_commander.py
└── src
```

5.2 部分介绍

`src/offboard_run`

`offboard_run` 目录包含了主要的 ROS 包和相关的代码文件。该目录下的内容用于实现无人机的导航和控制功能。

- **scripts:** 该目录包含Python脚本：
 - `vtol_commander.py`: 该脚本用于固定翼无人机 A 阶段一和阶段二的起飞、搜索巡航、返回并降落的控制，通过ROS实现。
 - `detect_stage1.py`: 该脚本用于固定翼无人机 A 阶段二的视觉识别，使用YOLOv8进行目标识别与坐标转换，将目标人物的坐标发送到指定话题。
 - `iris_commander.py`: 该脚本用于四旋翼无人机 B 阶段三和阶段四的起飞，接近健康人员目标、接近重伤人员目标、返回并降落的控制，通过ROS实现。

- `detect_white.py`: 该脚本用于四旋翼无人机 B 阶段三的健康人员的视觉识别和精确降落控制，在接近健康人员目标以后，通过视觉识别的结果进行轨迹采样并预测，得到降落平台的坐标后，下降到指定高度并发布话题。
- `detect_red.py`: 该脚本用于四旋翼无人机 B 阶段三的重伤人员的视觉识别和精确降落控制，在接近重伤人员目标以后，通过视觉识别的结果进行目标接近与降落，下降到指定高度并发布话题。
- **CMakeLists.txt**: 用于配置项目的CMake构建系统，该文件列出了构建项目所需的源文件和依赖项。
- **package.xml**: 描述ROS包的元数据文件。该文件包含包的名称、版本、维护者、依赖项等信息。
- **models**: 该目录包含了 `YOLOv8` 模型的相关模型文件，用于目标检测。
- **launch.sh**: 该脚本用于一键启动整个项目。
- **score_check.py**: 该脚本用于模拟评分。

6. 开发过程

6.1 整体流程

开始项目的开发以前我们先配置了环境，并学习了ros的相关语法知识。为了了解项目情况我们先初步尝试了无人机的飞行，在网络上搜索相关的控制脚本，首先成功实现了无人机解锁，并飞到20米的功能。在了解项目大致内容和目标后，我们将固定翼无人机的功能分为导航控制和视觉识别两个模块，将四旋翼无人机的功能分为导航控制和精确降落两个模块，接着完成两个无人机四个阶段的话题衔接。

6.2 项目分工

我们的项目按照任务类型——无人机飞行控制与视觉识别两个大方向进行划分，各自由两名同学负责两个模块的开发与测试，最后整合。

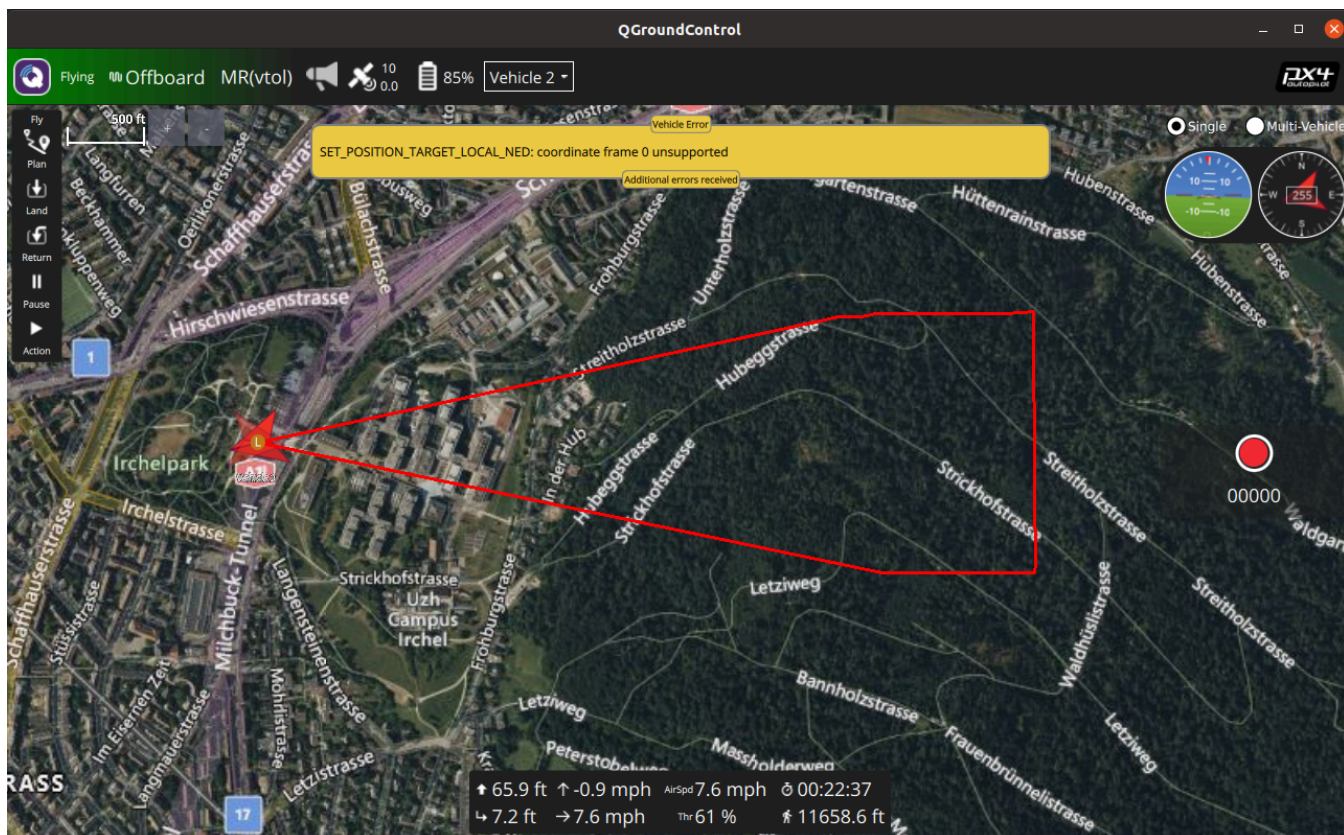
- 阶段一：固定翼无人机起飞，飞行控制，避障与搜索路径规划；
- 阶段二：视觉识别，坐标转换；视觉模型的数据采集与训练；
- 阶段三：四旋翼无人机起飞，飞行控制，避障路径规划，接近目标；
- 阶段四：根据视觉识别返回的物体坐标，实现精确降落；视觉模型的数据采集与训练。

7. 文件说明

7.1 `vtol_commander.py`（阶段一）

`vtol_commander.py` 用于固定翼无人机的导航控制，根据GPS坐标点绕过禁飞区到达目标区域上空，激活第一阶段的视觉识别进程并进行目标搜索。

7.1.1 整体路线预览



7.1.2 起飞降落逻辑

对于固定翼无人机，先使用多旋翼模式起飞到20米高度，后切换为固定翼模式完成所有任务，最后回到旋翼模式区时切换回多旋翼模式并降落并锁桨。

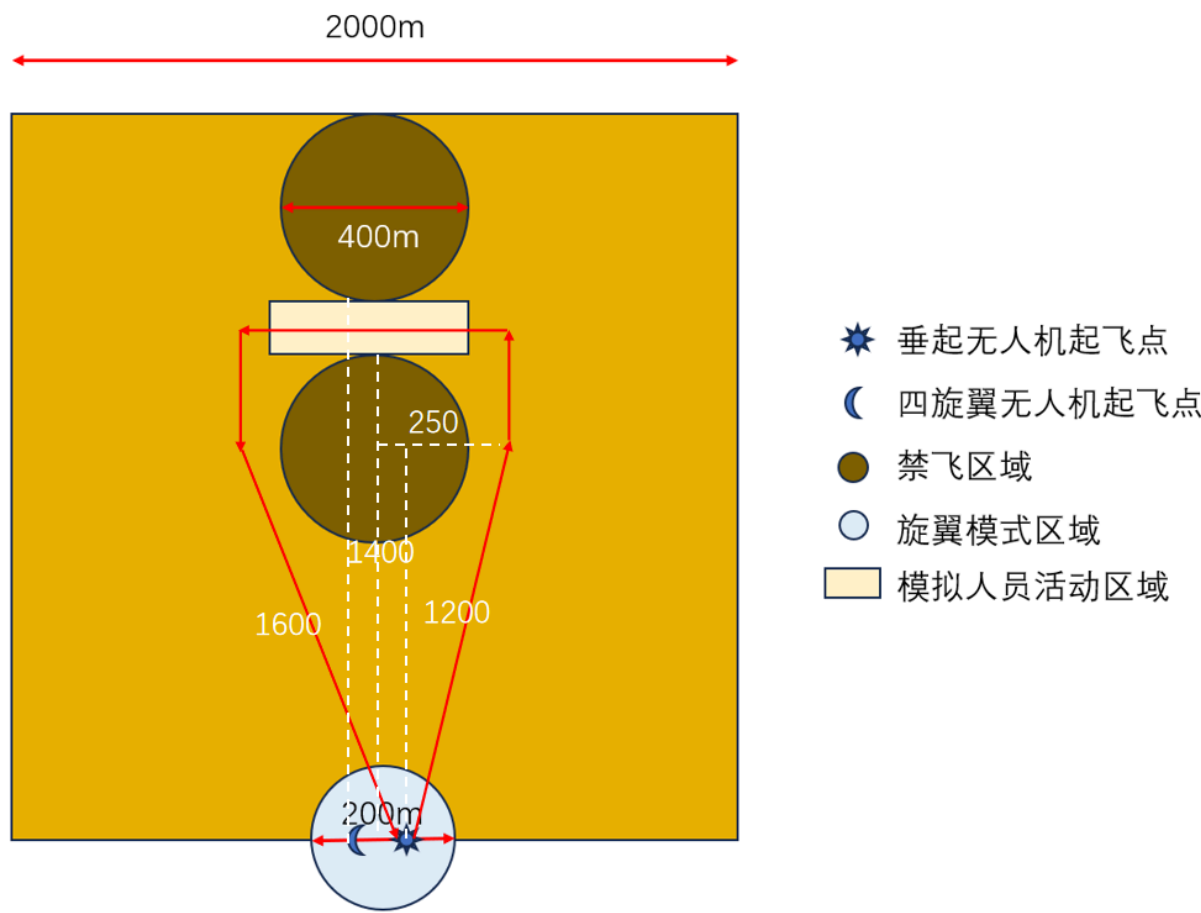
```
jane@ubuntu: ~/catkin_ws/src/offboard_run/scripts

[终端8] VTOL 控制脚本 (vtol_commander.py) 启动中...
[INFO] [1755148453.312362, 10255.008000]: Commander node initialized.
[INFO] [1755148453.315318, 10255.010000]: Waiting for first position message...
[INFO] [1755148453.378914, 10255.060000]: Position received.
[INFO] [1755148453.380373, 10255.061000]: Waiting for FCU connection...
[INFO] [1755148453.381767, 10255.062000]: FCU connected.
[INFO] [1755148453.382638, 10255.063000]: Attempting to activate OFFBOARD mode and arm...
[INFO] [1755148469.059576, 10260.115000]: OFFBOARD mode enabled
[INFO] [1755148488.763528, 10265.177000]: Vehicle armed
[INFO] [1755148489.963801, 10265.513000]: Vehicle is in OFFBOARD mode and armed.
[INFO] [1755148489.965705, 10265.513000]: Takeoff to height: 20.00
[INFO] [1755148529.651271, 10276.514000]: Takeoff complete. Hovering at 19.51 meters.
[INFO] [1755148541.368883, 10279.514000]: Switching to FIXED-WING mode...
[INFO] [1755148543.039661, 10280.015000]: Rotor mode zone center set at (-0.05, -0.04)
[INFO] [1755148543.042176, 10280.016000]: switching to fixed wing mode...
[INFO] [1755148545.634611, 10281.022000]: --- Starting Cruise Mission ---
[INFO] [1755148545.647766, 10281.022000]: change to altitude: 40.00
```

```
jane@ubuntu: ~/catkin_ws/src/offboard_run/scripts
[INFO] [1755149439.857883, 10534.905000]: Step 1: Returning to rotor zone center in fixed-wing mode.
[INFO] [1755149439.860758, 10534.905000]: Moving to the target position (-0.05, -0.04, 20.01)
[INFO] [1755149844.431043, 10642.755000]: Move command finished for target (-3.16, -0.55, 20.68).
[INFO] [1755149844.435367, 10642.760000]: Step 2: Arrived at zone center. Switching to multirotor mode.
[INFO] [1755149844.443886, 10642.760000]: Switching to MULTIROTOR mode...
[INFO] [1755149864.432760, 10648.262000]: Step 3: Commanding automatic land.
[INFO] [1755149864.434615, 10648.263000]: Commanding LAND
[INFO] [1755149864.453910, 10648.273000]: Land command sent successfully.
[INFO] [1755149864.460965, 10648.275000]: Waiting for landing and disarm...
[INFO] [1755149945.984092, 10671.282000]: Landed and disarmed successfully. Mission complete.
[INFO] [1755149945.988326, 10671.283000]: Commander node terminated.
GAZEBO_PLUGIN_PATH : /home/jane/PX4_Firmware/build/px4_sitl_default/build_gazebo:/home/jane/PX4_Firmware/build/px4_sitl_default/build_gazebo
GAZEBO_MODEL_PATH : /home/jane/PX4_Firmware//Tools/sitl_gazebo/models:/home/jane/PX4_Firmware//Tools/sitl_gazebo/models
LD_LIBRARY_PATH : /home/jane/catkin_ws/devel/lib:/opt/ros/noetic/lib:/home/jane/PX4_Firmware/build/px4_sitl_default/build_gazebo:/home/jane/PX4_Firmware/build/px4_sitl_default/build_gazebo
```

7.1.3 避障策略

在搜索任务中存在禁飞区，采取的方式是当无人机经过时保持一定距离地绕开；考虑到有房屋和树木的阻挡，在飞行过程中设置了较高的飞行速度避免碰撞，示意图如下。



7.2 detect_stage1.py (阶段二)

`detect_stage1.py` 文件实现了利用vtol无人机遥感数据进行目标识别并定位，并根据不同的目标类型对数据进行特定处理

- 数据读取：使用消息队列保存同一时刻的图像和位置信息避免数据阻塞，由识别线程统一处理
- 识别功能：利用YOLOv8这类预训练模型对遥感数据进行目标识别和标记
- 坐标计算：首先通过可信度筛选出高可信度的标记，通过建立的遥感坐标转换模型来计算物体的具体坐标
- 数据处理：对静止目标坐标进行均值滤波后进行发布，提高位置可信度；对运动目标则选择靠近遥感图像中心位置的坐标进行发布，以减少计算误差

PS: 遥感坐标转换模型。由像素坐标得到目标相对于相机系的方向向量，然后再由相机系转为机体系，机体系转为世界系坐标，根据地面高度为0的假设和线面相交的几何关系最终计算出目标位置。因为是在运动的平台上进行遥感计算，同时还需要修正数据因采样时间差造成的误差

1. 像素坐标转相机方向向量，其中 K 为相机内参， u, v 为目标像素坐标

$$\mathbf{p}_{\text{cam}} = K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}.$$

2. 相机坐标系到机体坐标系， $\mathbf{t}_{\text{cam} \rightarrow \text{body}}$ 为相对偏移

$$\mathbf{p}_{\text{body}} = R_{\text{cam} \rightarrow \text{body}} \mathbf{p}_{\text{cam}} + \mathbf{t}_{\text{cam} \rightarrow \text{body}}.$$

3. 机体坐标系到世界坐标系

$$\mathbf{p}_{\text{target}} = \mathbf{p}_{\text{UAV}} + R_{\text{body} \rightarrow \text{world}} \mathbf{p}_{\text{body}}.$$

4. 三维坐标计算，计算 `ray` 与地面 `z=ground_z` 的交点 `t= -z/dz`

$$t = (\text{ground}_z - \text{cam}_z) / \text{ray}_{\text{world}}[2]$$
$$P = \text{Cam}_{\text{position}} + t * \text{ray}_{\text{world}}$$

5. 误差修正，计算误差 v_{error} 并修正，得最终坐标

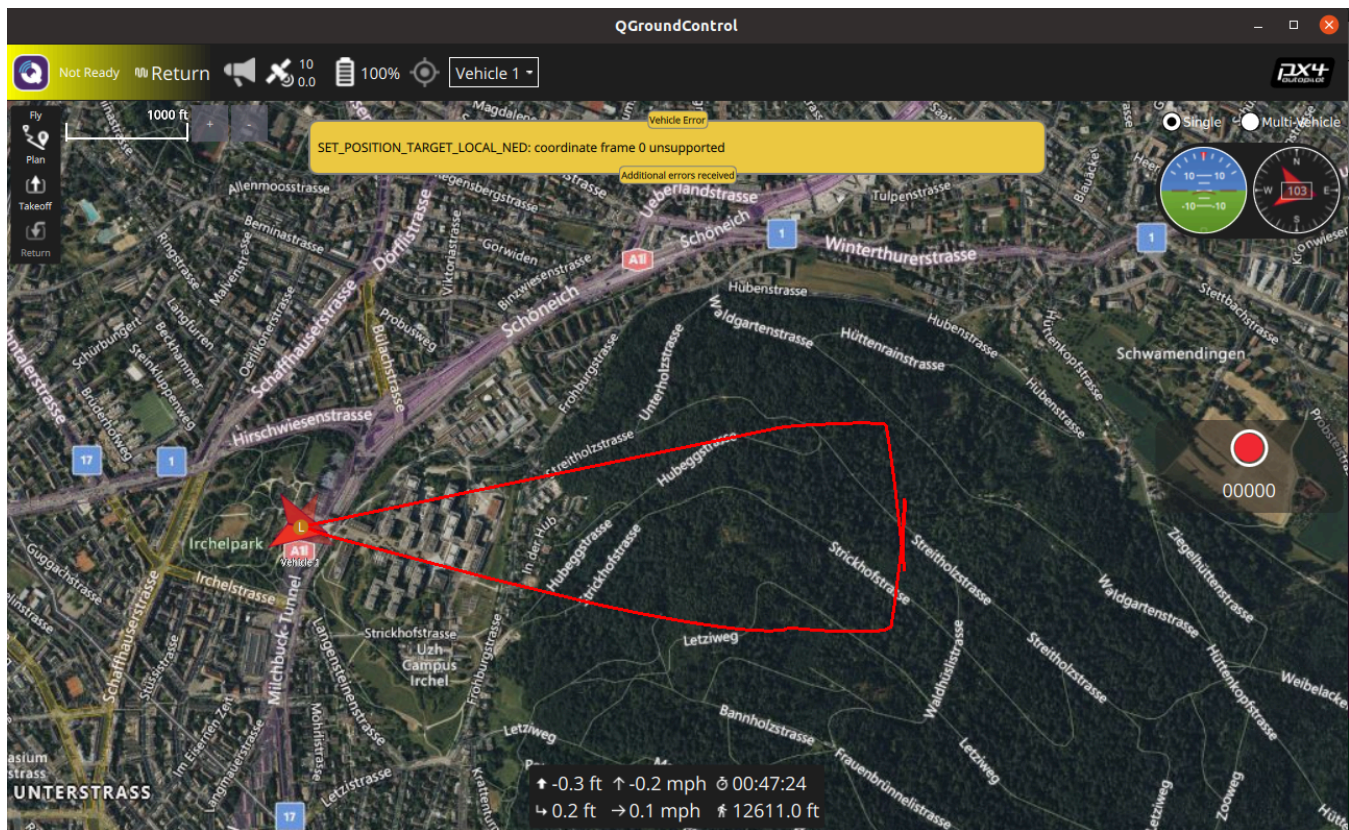
$$P = P + v_{\text{error}}$$

7.3 iris_commander.py (阶段三)

`iris_commander.py` 用于四旋翼无人机的导航控制，根据阶段二识别到的坐标位置绕过禁飞区到达目标区域上空，激活第四阶段的视觉识别进程并进行精确定位与降落。

7.3.1 整体路线预览

由于目标人物的生成位置是随机的，此处路线仅为示例

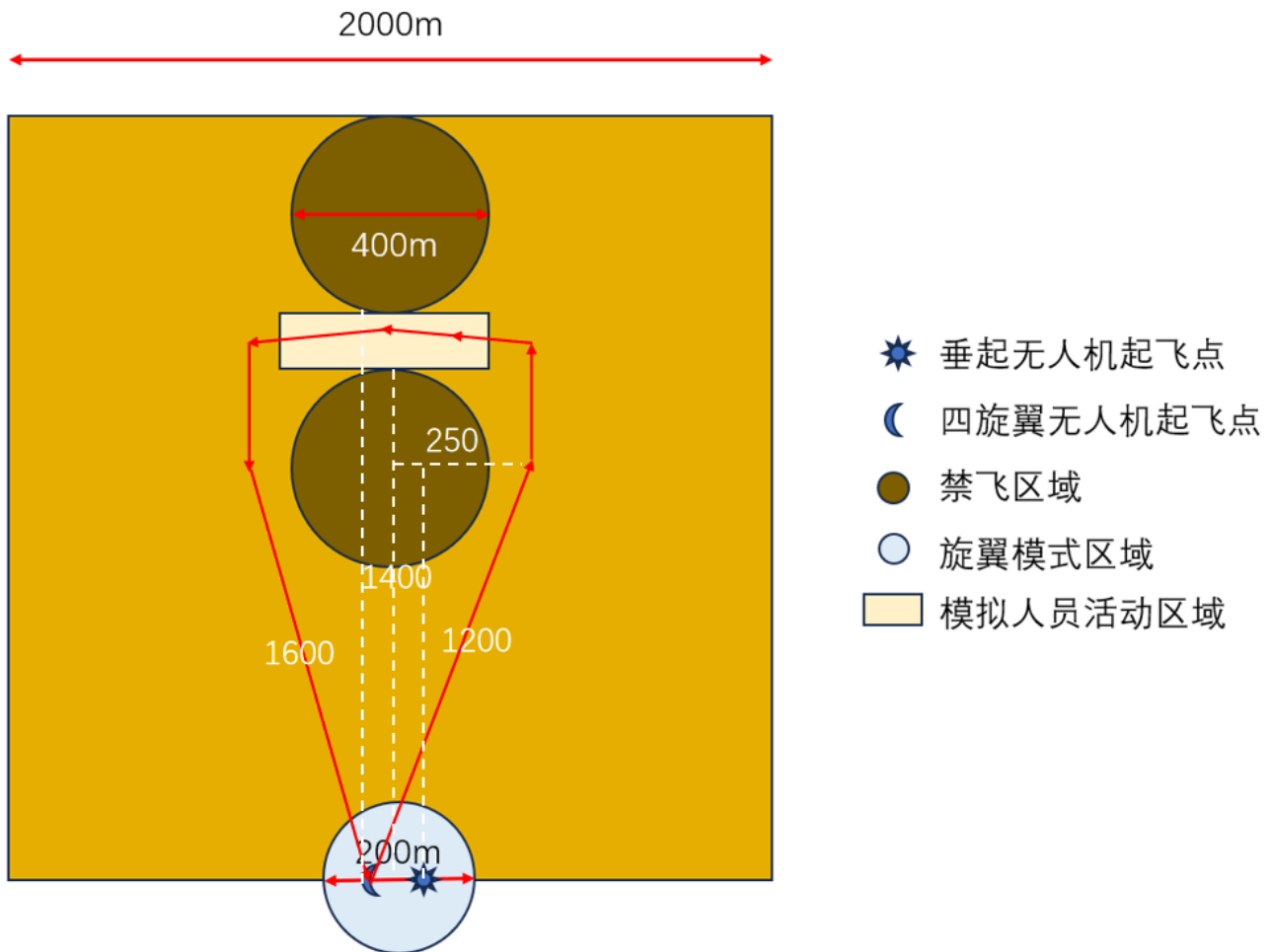


7.3.2 控制逻辑

对于四旋翼无人机只需要直接使用ROS话题控制无人机飞行即可，不再需要模式切换且可以自由悬停移动，可以更加灵活地控制以实现精确降落。

7.3.3 避障策略

采用相同的逻辑，示意图如下：



7.4 detect_white.py & detect_red.py（阶段四）

作用是无人机到达目标区域后进行递进侦查，整体框架分为目标识别进程和控制线程

- 目标识别线程：
 - 同时保存遥感图像和飞机位姿信息于队列中，利用YOLOv8类的视觉识别模型进行目标标注，并缩小识别范围以提高精确度，在选取合理可信度的标注信息通过遥感视线模型计算目标位置，交由无人机控制线程对目标进行递进侦查。因为飞机与目标都在做运行，坐标计算后需要进行相对误差修正，同时还需要控制数据的实时同步处理与发布。
- UAV控制线程：
 - 对于移动中的健康人员：采用搜索—建模—降落—等待的控制逻辑
 - **SEARCHING:** 首先移动到阶段二中搜索得到的坐标位置，并下降到一定高度；
 - **MODELING:** 接着根据视觉识别线程得到的坐标信息开始采样，建模拟合目标人物的运动轨迹；
 - **INTERCEPTING:** 根据计算得到的运动轨迹选择一个拦截点，无人机移动到这个点的上方
 - **DESCENDING_TO_WAIT:** 下降到发布话题的要求高度0.7米
 - **WAITING_FOR_TARGET:** 等待目标通过即发布坐标
 - 对于重伤人员：采用识别——对准——下降的控制逻辑
 - **SEARCHING:** 根据视觉识别算法得到的目标坐标进行水平位置的微调，当无人机恰好移动到目标正上方时进入下一状态；
 - **ALIGNING:** 对准后允许下降一段距离

- 重复上述过程，直到无人机下降到发布话题的要求高度0.5米

8. 关键代码及注释

8.1 detect_stage1.py代码及注释

```
# detect_stage1.py
import rospkg
import os
import rospy
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped, Pose
from cv_bridge import CvBridge
import cv2
from ultralytics import YOLO
import numpy as np
from tf.transformations import quaternion_matrix
from scipy.spatial.transform import Rotation as R
from std_msgs.msg import String, Bool
from collections import deque
import threading
from queue import Queue, Empty

class ObjectLocalizationNode:
    def __init__(self):
        rospy.init_node('object_localization_node_stage_1', anonymous=True)

        self.bridge = CvBridge()

        rospack = rospkg.RosPack()
        pkg_path = rospack.get_path('offboard_run')
        model_path = os.path.join(pkg_path, 'models', 'stage1.pt')
        self.model = YOLO(model_path)

        self.camera_info = None
        self.drone_pose = None
        self.can_start = False
        self.ending = False
        self.image_rate = rospy.Rate(10)

        # 队列保存检测结果
        self.target_queues = {
            "red": deque(maxlen=10),
            "yellow": deque(maxlen=10),
            "white": deque(maxlen=10)
        }

        self.target_pause_triggered = {
            "red": False,
            "yellow": False,
            "white": False
        }
```

```

}

# 发布平滑后的目标位置，显然，目标会移动，yellow & white坐标是错的
self.pose_publishers = {
    "red": rospy.Publisher('/zhihang2025/first_man/pose', Pose, queue_size=10),
    "yellow": rospy.Publisher('/zhihang2025/second_man/pose', Pose, queue_size=10),
    "white": rospy.Publisher('/zhihang2025/third_man/pose', Pose, queue_size=10)
}

# 订阅相机图像和信息
rospy.Subscriber('/standard_vtol_0/camera/image_raw', Image, self.image_callback,
queue_size=10)
rospy.Subscriber('/standard_vtol_0/camera/camera_info', CameraInfo,
self.camera_info_callback, queue_size=10)
rospy.Subscriber('/standard_vtol_0/mavros/local_position/pose', PoseStamped,
self.drone_pose_callback, queue_size=1)
rospy.Subscriber('/standard_vtol_0/waypoint_reached', Bool, self._reached_cb)
rospy.Subscriber('/standard_vtol_0/search_completed', Bool, self._ending_cb)
self._init_tf() # 初始化相机到机体坐标变换矩阵

# 发布3D目标位置和种类
self.status_pub = rospy.Publisher('/zhihang2025/detection_status', Bool,
queue_size=1)

rospy.loginfo("Object Localization Node Initialized.")
self.image_queue = Queue(maxsize=3)
self.processing_thread = threading.Thread(target=self.detection_worker)
self.processing_thread.daemon = True
self.processing_thread.start()
rospy.loginfo("start detection thread")

def _init_tf(self):
    self.T_cam2body = np.eye(4)
    self.T_cam2body[:3, :3] = np.array([
        [ 0, -1,  0],
        [-1,  0,  0],
        [ 0,  0, -1]
    ])
    self.T_cam2body[:3, 3] = [0.0, 0.0, -0.03] # 相机在无人机下方 3cm, check .sdf

def camera_info_callback(self, msg):
    # 提取相机内参矩阵 K 和畸变系数 D
    self.camera_info = msg
    self.camera_matrix = np.array(msg.K).reshape((3, 3))
    self.dist_coeffs = np.array(msg.D)

def drone_pose_callback(self, msg):
    self.drone_pose = msg

def _reached_cb(self, msg):
    if msg.data:
        self.can_start = True
        rospy.loginfo("Reached waypoint, starting to detect")

```

```

def _ending_cb(self, msg):
    if msg.data:
        self.ending = True
        rospy.loginfo("Mission end...")

def image_callback(self, msg):
    if not self.can_start:
        return
    if self.camera_info is None or self.drone_pose is None:
        rospy.logdebug("waiting for camera info and drone pose...")
        return

    # 检查队列数据类型, 确保入队的是(msg, self.drone_pose)
    try:
        # 深拷贝当前的drone_pose, 避免后续被异步修改
        import copy
        pose_copy = copy.deepcopy(self.drone_pose)
        if not self.image_queue.full():
            self.image_queue.put((msg, pose_copy))
        else:
            try:
                self.image_queue.get_nowait()
                self.image_queue.put((msg, pose_copy))
            except Exception as e:
                rospy.logwarn(f"Queue put error: {e}")
    except Exception as e:
        rospy.logerr(f"Image callback error: {e}", exc_info=True)

```

```

def detection_worker(self):
    while not rospy.is_shutdown():
        try:
            msg, drone_pose = self.image_queue.get(timeout=1.0)
            cv_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
            # 执行YOLOv8检测
            results = self.model(source=cv_image, verbose=False)
            found_target = False
            for r in results:
                boxes = r.boxes
                for box in boxes:
                    # 获取2D边界框和类别信息
                    x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
                    conf = float(box.conf.item())
                    cls = int(box.cls.item())
                    object_type = self.model.names[cls]

                    if conf > 0.7:
                        # 获取边界框中心点 (u, v)
                        u_pixel = (x1 + x2) / 2
                        v_pixel = (y1 + y2) / 2

                        target_position = self.get_target_position(u_pixel, v_pixel,
drone_pose.pose)

```

```

        found_target = True
        rospy.loginfo(f"Detected {object_type} at 3D position (World):
{target_position}")

        # 分类保存到队列
        if object_type in self.target_queues:
            self.target_queues[object_type].append(target_position)
            self.publish_smoothed_position(object_type, target_position,
u_pixel, v_pixel)

        # 绘制边界框
        cv2.rectangle(cv_image, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.putText(cv_image, f"{object_type} {conf:.2f} pose:
{target_position}", (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

        # 显示图像（仅在检测到目标时）
        if found_target:
            scale = 0.5 # 缩小一半
            resized_image = cv2.resize(cv_image, (0, 0), fx=scale, fy=scale)
            cv2.imshow("YOLOv8 Detection", resized_image)
            cv2.waitKey(1)
        except Empty:
            continue
        except Exception as e:
            rospy.logerr(f"Detection error: {e}", exc_info=True)
            self.image_rate.sleep()

    def publish_smoothed_position(self, object_type, target_position, u, v):
        queue = self.target_queues[object_type]
        if object_type == "red" and len(queue) > 5: # and u > 320 and u < 960 and v > 180
and v < 540
            smoothed_position = np.mean(queue, axis=0)
            pose_msg = Pose()
            pose_msg.position.x = smoothed_position[0] - 3
            pose_msg.position.y = smoothed_position[1]
            pose_msg.position.z = smoothed_position[2]
            self.pose_publishers[object_type].publish(pose_msg)
            rospy.loginfo(f"Published smoothed position for {object_type}:
{smoothed_position}")

            if object_type == "yellow" and u > 320 and u < 960 and v > 180 and v < 540: # 保证黄
色和红色识别的时候在整个视界的中心矩形区域内，减少计算误差
                pose_msg = Pose()
                pose_msg.position.x = target_position[0] - 3 + 1.23
                pose_msg.position.y = target_position[1]
                pose_msg.position.z = target_position[2]
                self.pose_publishers[object_type].publish(pose_msg)
                rospy.loginfo(f"Published position for {object_type}: {target_position}")

            if object_type == "white" and u > 320 and u < 960 and v > 180 and v < 540:
                pose_msg = Pose()
                pose_msg.position.x = target_position[0] - 3 + 1.23
                pose_msg.position.y = target_position[1]

```

```

        pose_msg.position.z = target_position[2]
        self.pose_publishers[object_type].publish(pose_msg)
        rospy.loginfo(f"Published position for {object_type}: {target_position}")

def normalize(self, v):
    return v / np.linalg.norm(v)

def pixel_to_camera_ray(self, u, v):
    pixel_h = np.array([u, v, 1.0]) # 齐次像素
    ray_cam = np.linalg.inv(self.camera_matrix) @ pixel_h
    return self.normalize(ray_cam)

def camera_ray_to_world(self, ray_cam, orientation):
    # 相机坐标 → 机体坐标
    ray_body = self.T_cam2body[:3, :3] @ ray_cam
    # 机体坐标 → 世界坐标
    q = orientation
    R_world_body = quaternion_matrix([q.x, q.y, q.z, q.w])[:3, :3]
    ray_world = R_world_body @ ray_body
    return self.normalize(ray_world)

def get_target_position(self, u, v, pose, ground_z=0.0):
    # 根据T时刻位姿和图像信息计算指向目标的方向向量并且做坐标转换
    if self.camera_info is None or pose is None:
        rospy.logwarn("waiting for camera info and drone pose...")
        return None

    # Step 1: 像素坐标转相机方向向量
    ray_cam = self.pixel_to_camera_ray(u, v)

    # Step 2: 相机方向转世界坐标方向向量
    ray_world = self.camera_ray_to_world(ray_cam, pose.orientation)

    # Step 3: 相机位置（考虑相机相对无人机机体的偏移）
    cam_offset = self.T_cam2body[:3, 3] # 平移向量
    R_world_body = quaternion_matrix([pose.orientation.x,
                                      pose.orientation.y,
                                      pose.orientation.z,
                                      pose.orientation.w])[:3, :3]
    drone_position = np.array([pose.position.x, pose.position.y, pose.position.z])
    v1 = np.array([2.3, 0.4, 1.3]) # local相对world的偏移
    v2 = np.array([0, 0, -0.05]) # 相机相对于机体的偏移
    drone_position = drone_position + v1 + v2 # 假设无人机位置偏移
    # 计算相机在世界坐标系中的位置
    cam_position = drone_position + R_world_body @ cam_offset

    # Step 4: 计算 ray 与地面 Z=ground_z 的交点 t = -z / dz
    t = (ground_z - cam_position[2]) / ray_world[2]
    if t < 0:
        rospy.logwarn("Ray does not intersect ground plane.")
        return None

    target_world = cam_position + t * ray_world

```

```

v3 = np.array([1.2, -0.5, 0])
target_world -= v3 # 人物相对于靶心的偏移，这里主要是和算分的脚本对齐，并不知道实际算分的时候是
怎么样的
return target_world

def run(self):
    rospy.spin()

if __name__ == '__main__':
    try:
        node = ObjectLocalizationNode()

        rospy.loginfo("waiting for waypoint reached...")
        rate = rospy.Rate(1)
        while not rospy.is_shutdown() and not node.can_start:
            rospy.loginfo_throttle(10, "waiting for signal...")
            rate.sleep()
        rospy.loginfo("Signal received! Starting to detect...")
        while not rospy.is_shutdown():
            if node.ending:
                rospy.loginfo("Ending signal received. Shutting down the node.")
                rospy.signal_shutdown("Mission part 1 completed, shutdown signal received")
                break
            rate.sleep()

    except rospy.ROSInterruptException:
        rospy.loginfo("Node interrupted by user")
    except Exception as e:
        rospy.logfatal(f"An unhandled error occurred: {e}", exc_info=True)
    finally:
        rospy.loginfo("Object Localization node is shutting down.")
        cv2.destroyAllWindows()

```

8.2 vtol_commander.py代码及注释

```

# vtol_commander.py
import rospy
import tf
import math
from geometry_msgs.msg import PoseStamped, Pose, Quaternion, Twist, TwistStamped
from mavros_msgs.msg import State
from mavros_msgs.srv import CommandBool, SetMode
from sensor_msgs.msg import Image
from std_msgs.msg import String, Bool
from cv_bridge import CvBridge, CvBridgeError
import cv2
import time

class Commander:
    def __init__(self):

```

```

rospy.init_node("commander_node")

self.current_state = State()
self.current_pose = Pose()
self.current_yaw = 0.0
self.position_received = False
self.bridge = CvBridge()

self.is_fixed_wing = False
self.fixed_wing_start_point = None
self.rotor_zone_radius = 100.0
self.searched = False
self.is_paused = False

# --- 订阅者与发布者 ---
rospy.Subscriber("standard_vtol_0/mavros/state", State, self._state_cb)
rospy.Subscriber("standard_vtol_0/mavros/local_position/pose", PoseStamped,
self._pose_cb)
rospy.Subscriber("standard_vtol_0/camera/image_raw", Image, self._image_callback)

self.setpoint_pos_pub =
rospy.Publisher("standard_vtol_0/mavros/setpoint_position/local", PoseStamped,
queue_size=10)
self.xtdrone_vel_pub = rospy.Publisher("xtdrone/standard_vtol_0/cmd_vel_flu", Twist,
queue_size=10)
self.xtdrone_cmd_pub = rospy.Publisher("xtdrone/standard_vtol_0/cmd", String,
queue_size=10)
self.waypoint_pub = rospy.Publisher("standard_vtol_0/waypoint_reached", Bool,
queue_size=10)
self.ending_pub = rospy.Publisher("standard_vtol_0/search_completed", Bool,
queue_size=10)
self.part1_completed_pub = rospy.Publisher("/part1_completed", Bool, queue_size=10)

# --- 服务客户端 ---
rospy.wait_for_service("standard_vtol_0/mavros/cmd/arming")
self.arming_client = rospy.ServiceProxy("standard_vtol_0/mavros/cmd/arming",
CommandBool)
rospy.wait_for_service("standard_vtol_0/mavros/set_mode")
self.set_mode_client = rospy.ServiceProxy("standard_vtol_0/mavros/set_mode",
SetMode)
rospy.Subscriber('/zhihang2025/detection_status', Bool,
self.detection_status_callback)

rospy.loginfo("Commander node initialized.")

def _switch_to_fixed_wing(self):
    if not self.is_fixed_wing:
        rospy.loginfo("Switching to FIXED-WING mode...")
        cmd = String()
        cmd.data = "plane"
        # 多发布几次确保指令被接收
        for _ in range(5):
            self.xtdrone_cmd_pub.publish(cmd)

```

```

        rospy.sleep(0.1)
        self.is_fixed_wing = True
        # 记录切换点作为旋翼模式区的中心
        if self.fixed_wing_start_point is None:
            self.fixed_wing_start_point = self.current_pose.position
            rospy.loginfo("Rotor mode zone center set at (%.2f, %.2f)" %
(self.fixed_wing_start_point.x, self.fixed_wing_start_point.y))

def _switch_to_multirotor(self):
    if self.is_fixed_wing:
        rospy.loginfo("Switching to MULTIROTOR mode...")
        cmd = String()
        cmd.data = "multirotor"
        for _ in range(5):
            self.xtdrone_cmd_pub.publish(cmd)
            rospy.sleep(0.1)
        self.is_fixed_wing = False

# --- 回调函数 ---
def _state_cb(self, msg):
    self.current_state = msg

def _pose_cb(self, msg):
    self.current_pose = msg.pose
    self.position_received = True

    orientation_q = self.current_pose.orientation
    _, _, self.current_yaw = tf.transformations.euler_from_quaternion(
        [orientation_q.x, orientation_q.y, orientation_q.z, orientation_q.w])

def _image_callback(self, msg):
    try:
        cv_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        scale = 0.5
        resized_image = cv2.resize(cv_image, (0, 0), fx=scale, fy=scale)
        cv2.imshow("Standard_vtol Camera", resized_image)
        cv2.waitKey(1)
    except CvBridgeError as e:
        rospy.logerr(e)

def detection_status_callback(self, msg):
    rospy.loginfo(f"Received detection status: {'PAUSED' if msg.data else 'RESUMED'}")
    self.is_paused = msg.data

def _distance(self, p1, p2):
    return math.sqrt(math.pow(p1.x - p2.x, 2) + math.pow(p1.y - p2.y, 2))

def publish_velocity(self, forward=0.0, leftward=0.0, upward=0.0, angular_z=0.0):
    """发布速度指令 (机体坐标系: 前左上)"""
    twist_msg = Twist()
    twist_msg.linear.x = forward
    twist_msg.linear.y = leftward
    twist_msg.linear.z = upward

```

```

twist_msg.angular.z = angular_z
self.xtdrone_vel_pub.publish(twist_msg)

def activate_offboard_and_arm(self):
    """
    激活 OFFBOARD 模式并解锁无人机。
    这是一个阻塞方法，直到成功或ROS关闭。
    @return: 成功返回 True，否则返回 False。
    """
    rate = rospy.Rate(20.0)

    rospy.loginfo("Attempting to activate OFFBOARD mode and arm...")

    pose = PoseStamped()

    # 等待位置信息
    while not rospy.is_shutdown() and not self.position_received:
        rospy.logwarn_throttle(2, "waiting for position information...")
        rate.sleep()

    pose.pose = self.current_pose

    last_request = rospy.Time.now()

    while not rospy.is_shutdown():
        current_time = rospy.Time.now()

        # 检查并设置模式
        if self.current_state.mode != "OFFBOARD" and (current_time - last_request >
rospy.Duration(5.0)):
            try:
                if self.set_mode_client(custom_mode='OFFBOARD').mode_sent:
                    rospy.loginfo("OFFBOARD mode enabled")
            except rospy.ServiceException as e:
                rospy.logerr("Service call for OFFBOARD failed: %s" % e)
            last_request = current_time

        # 检查并解锁
        elif not self.current_state.armed and (current_time - last_request >
rospy.Duration(5.0)):
            try:
                if self.arming_client(True).success:
                    rospy.loginfo("Vehicle armed")
                    self.xtdrone_cmd_pub.publish("ARM")
            except rospy.ServiceException as e:
                rospy.logerr("Service call for arming failed: %s" % e)
            last_request = current_time

        # 检查是否成功
        if self.current_state.mode == "OFFBOARD" and self.current_state.armed:
            rospy.loginfo("Vehicle is in OFFBOARD mode and armed.")
            self.xtdrone_cmd_pub.publish("ARM")
            return True

```

```

        # 持续发布设定点以保持连接
        pose.header.stamp = rospy.Time.now()
        pose.pose = self.current_pose
        self.setpoint_pos_pub.publish(pose)
        rate.sleep()

    return False

def disarm(self):
    rospy.loginfo("Commanding DISARM")
    try:
        if self.arming_client(False).success:
            rospy.loginfo("Vehicle disarmed successfully.")
    except rospy.ServiceException as e:
        rospy.logerr("Service call for disarming failed: %s" % e)

# --- `takeoff` 函数只负责起飞 ---
def takeoff(self, height):
    rate = rospy.Rate(20.0)

    rospy.loginfo("Waiting for first position message...")
    while not rospy.is_shutdown() and not self.position_received: rate.sleep()
    rospy.loginfo("Position received.")
    rospy.loginfo("Waiting for FCU connection...")
    while not rospy.is_shutdown() and not self.current_state.connected: rate.sleep()
    rospy.loginfo("FCU connected.")

    if not self.activate_offboard_and_arm():
        rospy.logfatal("Failed to activate OFFBOARD and arm. Aborting takeoff.")
        return

    rospy.loginfo("Takeoff to height: %.2f" % height)
    takeoff_pose = PoseStamped()
    takeoff_pose.pose.position.x = self.current_pose.position.x
    takeoff_pose.pose.position.y = self.current_pose.position.y
    takeoff_pose.pose.position.z = height

    while not rospy.is_shutdown():
        takeoff_pose.header.stamp = rospy.Time.now()
        self.setpoint_pos_pub.publish(takeoff_pose)

        if abs(self.current_pose.position.z - height) < 0.5:
            rospy.loginfo("Takeoff complete. Hovering at %.2f meters." %
self.current_pose.position.z)
            # 起飞完成后, 继续发送指令以保持悬停, 然后返回
            for _ in range(60):
                takeoff_pose.header.stamp = rospy.Time.now()
                self.setpoint_pos_pub.publish(takeoff_pose)
                rate.sleep()

            return
        rate.sleep()

```

```

# --- 改变高度的函数 ---
def change_altitude(self, height):
    rospy.loginfo("change to altitude: %.2f" % height)
    rate = rospy.Rate(20.0)

    target_pose = PoseStamped()
    target_pose.pose.position.x = self.current_pose.position.x
    target_pose.pose.position.y = self.current_pose.position.y
    target_pose.pose.position.z = height

    while not rospy.is_shutdown():
        target_pose.header.stamp = rospy.Time.now()
        self.setpoint_pos_pub.publish(target_pose)

        if abs(self.current_pose.position.z - height) < 0.5:
            rospy.loginfo("Changing altitude to %.2f meters." %
self.current_pose.position.z)
            # 继续发送指令以保持悬停，然后返回
            for _ in range(100): # 悬停5秒
                target_pose.header.stamp = rospy.Time.now()
                self.setpoint_pos_pub.publish(target_pose)
                rate.sleep()
            return
        rate.sleep()

# --- `move` 函数控制移动 ---
def move(self, x, y, z, frame="world"):
    if not (self.current_state.mode == "OFFBOARD" and self.current_state.armed):
        rospy.loginfo("Attempting to reactivate OFFBOARD and arm...")
        if not self.activate_offboard_and_arm():
            rospy.logfatal("Failed to activate OFFBOARD and arm. Aborting move.")
            return

    target_pose = PoseStamped()
    target_pose.pose.position.x = x
    target_pose.pose.position.y = y
    target_pose.pose.position.z = z

    delta_x = target_pose.pose.position.x - self.current_pose.position.x
    delta_y = target_pose.pose.position.y - self.current_pose.position.y

    if math.sqrt(delta_x**2 + delta_y**2) > 1.0: # 避免在原地改变高度时改变偏航
        target_yaw_rad = math.atan2(delta_y, delta_x)
        target_quaternion = tf.transformations.quaternion_from_euler(0, 0,
target_yaw_rad)
        target_pose.pose.orientation = Quaternion(*target_quaternion)
    else:
        # 如果是垂直移动，保持当前偏航角
        target_pose.pose.orientation = self.current_pose.orientation

    rospy.loginfo("Moving to the target position (%.2f, %.2f, %.2f)" % (x, y, z))
    rate = rospy.Rate(20.0)

```

```

completion_radius = 15.0 if self.is_fixed_wing else 1.5

while not rospy.is_shutdown():
    target_pose.header.stamp = rospy.Time.now()
    self.setpoint_pos_pub.publish(target_pose)

    if self.is_paused:
        self.hold()
        rospy.loginfo_throttle(1, "Detection in progress, commander is PAUSED and
hovering.")

    # 只检查水平距离
    if self._distance(self.current_pose.position, target_pose.pose.position) <
completion_radius:
        start_time = rospy.Time.now()
        while rospy.Time.now() - start_time < rospy.Duration(2.0):
            self.setpoint_pos_pub.publish(target_pose)
            rate.sleep()

        rospy.loginfo("Move command finished for target (%.2f, %.2f, %.2f)." %
(self.current_pose.position.x, self.current_pose.position.y, self.current_pose.position.z))
        return
        rate.sleep()

    self.hold()
    rospy.loginfo("Move command finished.")

def move_with_velocity(self, x, y, z, target_speed):
    """
    使用速度控制向目标点移动
    @param x, y, z: 全局目标坐标
    @param target_speed: 飞向目标的期望速度 (m/s)
    """
    rospy.loginfo(f"Moving to ({x:.2f}, {y:.2f}, {z:.2f}) with speed {target_speed:.2f}
m/s...")
    rate = rospy.Rate(20.0)

    completion_radius = 15.0 if self.is_fixed_wing else 3.0

    while not rospy.is_shutdown():
        if not self.position_received:
            rate.sleep()
            continue

        if self.current_pose.position.x >= 1400 and self.searched == False:
            rospy.loginfo("Reaching waypoint, start to search")
            self.searched = True
            msg = Bool()
            msg.data = True
            self.waypoint_pub.publish(msg)

        # --- 核心控制逻辑 ---
        dx = x - self.current_pose.position.x

```

```

dy = y - self.current_pose.position.y
dz = z - self.current_pose.position.z

distance = math.sqrt(dx**2 + dy**2 + dz**2)

# 检查是否到达
if distance < completion_radius:
    rospy.loginfo("Target waypoint reached.")
    # 到达后发送悬停指令（零速度）
    self.xtdrone_vel_pub.publish(Twist())
    self.xtdrone_cmd_pub.publish(String("HOVER"))
    rospy.sleep(2) # 悬停稳定
    return

# 1. 计算全局坐标系下的速度向量
global_vel_x = (dx / distance) * target_speed
global_vel_y = (dy / distance) * target_speed
global_vel_z = (dz / distance) * target_speed

# 2. 将全局速度转换为机体坐标系下的速度（前、左、上）
forward_vel = global_vel_x * math.cos(self.current_yaw) + global_vel_y *
math.sin(self.current_yaw)
leftward_vel = -global_vel_x * math.sin(self.current_yaw) + global_vel_y *
math.cos(self.current_yaw)
upward_vel = global_vel_z

# 3. 发布Twist消息
twist_msg = Twist()
twist_msg.linear.x = forward_vel
twist_msg.linear.y = leftward_vel
twist_msg.linear.z = upward_vel

self.xtdrone_vel_pub.publish(twist_msg)
rate.sleep()

# --- 返航并在旋翼模式区降落 ---
def return_and_land(self):
    """
    命令无人机返回旋翼模式区中心，切换回多旋翼模式，然后降落。
    """
    if self.fixed_wing_start_point is None:
        rospy.logerr("Cannot return, fixed-wing start point was never set.")
        return

    rospy.loginfo("--- Starting Return, Switch and Land Sequence ---")

    # 步骤1：以固定翼模式返回旋翼区中心点附近（保持巡航高度）
    rospy.loginfo("Step 1: Returning to rotor zone center in fixed-wing mode.")
    self.move(self.fixed_wing_start_point.x, self.fixed_wing_start_point.y,
self.fixed_wing_start_point.z)

    # 步骤2：切换回多旋翼模式
    rospy.loginfo("Step 2: Arrived at zone center. Switching to multirotor mode.")

```

```

self._switch_to_multirotor()
rospy.sleep(5) # 给予充足时间让飞行状态稳定下来

# 步骤3: 执行自动降落
rospy.loginfo("Step 3: Commanding automatic land.")
self.land()

rospy.loginfo("waiting for landing and disarm...")
landing_start_time = rospy.Time.now()
while not rospy.is_shutdown() and self.current_state.armed:
    if rospy.Time.now() - landing_start_time > rospy.Duration(60.0):
        self.disarm()
        self.xtdrone_cmd_pub.publish("DISARM")
        break
    rospy.sleep(1)
self.xtdrone_cmd_pub.publish("DISARM")
if not self.current_state.armed:
    self.xtdrone_cmd_pub.publish("DISARM")
    rospy.loginfo("Landed and disarmed successfully. Mission complete.")
    msg = Bool()
    msg.data = True
    self.part1_completed_pub.publish(msg)
else:
    rospy.logerr("Mission finished, but failed to disarm the vehicle.")

def land(self):
    rospy.loginfo("Commanding LAND")
    try:
        if self.set_mode_client(custom_mode='AUTO.LAND').mode_sent:
            rospy.loginfo("Land command sent successfully.")
    except rospy.ServiceException as e:
        rospy.logerr("Service call failed: %s" % e)

def hold(self):
    rospy.loginfo("Switching to Position Hold mode (AUTO.LOITER)")
    try:
        if self.set_mode_client(custom_mode='AUTO.LOITER').mode_sent:
            rospy.loginfo("Switched to LOITER mode successfully.")
    except rospy.ServiceException as e:
        rospy.logerr("Service call for LOITER mode failed: %s" % e)

def start_to_search(self):
    if self.searched == False:
        rospy.loginfo("Reaching waypoint, start to search")
        self.searched = True
        msg = Bool()
        msg.data = True
        self.waypoint_pub.publish(msg)

def end_searching(self):
    rospy.loginfo("Searching mission ended...")
    msg = Bool()

```

```

        msg.data = True
        self.ending_pub.publish(msg)

if __name__ == "__main__":
    try:
        con = Commander()

        con.takeoff(20.0)

        con._switch_to_fixed_wing()
        rospy.loginfo("switching to fixed wing mode...")
        rospy.sleep(1)

        # 在固定翼模式下执行所有巡航任务
        rospy.loginfo("--- Starting Cruise Mission ---")
        con.change_altitude(40.0)
        con.move(1200, -250, 40.0)
        con.move(1495, -250, 40.0)

        rospy.loginfo("--- Starting Search Mission ---")
        con.change_altitude(10.0)
        con.start_to_search()
        con.move(1495, 250, 10)
        con.end_searching()

        con.change_altitude(40.0)
        con.move(1450, 250, 40)
        # 任务完成后，返回、切换模式并降落
        con.move(1200, 250, 40.0)
        con.return_and_land()

    except rospy.ROSInterruptException:
        pass
    finally:
        cv2.destroyAllWindows()
        rospy.loginfo("Commander node terminated.")

```

8.3 iris_commander.py代码及注释

```

# iris_commander.py
import rospy
import tf
import math
from geometry_msgs.msg import PoseStamped, Pose, Quaternion, Twist, TwistStamped
from mavros_msgs.msg import State, ExtendedState
from mavros_msgs.srv import CommandBool, SetMode
from sensor_msgs.msg import Image
from std_msgs.msg import String, Bool
from cv_bridge import CvBridge, CvBridgeError

```

```

import cv2

class XDroneController:
    def __init__(self):
        """
        初始化自主控制器节点
        """
        rospy.init_node("quadcopter_commander_node")

        self.current_state = State()
        self.current_pose = Pose()
        self.current_yaw = 0.0
        self.position_received = False
        self.bridge = CvBridge()
        self.can_start = False
        self.landed_state = ExtendedState.LANDED_STATE_UNDEFINED
        self.critical_man_pose = None
        self.healthy_man_pose = None
        self.critical_complete = False
        self.healthy_complete = False

        # subscriber
        rospy.Subscriber("/iris_0/mavros/state", State, self._state_cb)
        #rospy.Subscriber("/iris_0/mavros/local_position/pose", PoseStamped, self._pose_cb)
        rospy.Subscriber("/iris_0/mavros/vision_pose/pose", PoseStamped, self._pose_cb)
        rospy.Subscriber("/iris_0/mavros/extended_state", ExtendedState,
self._extended_state_cb)
        rospy.Subscriber("/iris_0/camera/image_raw", Image, self._image_callback)
        rospy.Subscriber("/part1_completed", Bool, self._start_cb)
        # publisher
        self.cmd_pub = rospy.Publisher("/xtdrone/iris_0/cmd", String, queue_size=1)
        self.cmd_vel_pub = rospy.Publisher("/xtdrone/iris_0/cmd_vel_flu", Twist,
queue_size=1)
        #self.setpoint_pos_pub = rospy.Publisher("/iris_0/mavros/setpoint_position/local",
PoseStamped, queue_size=10)
        self.waypoint_pub = rospy.Publisher("/iris_0/waypoint_reached", Bool, queue_size=10)
        self.part2_completed_pub = rospy.Publisher("/part2_completed", Bool, queue_size=10)

        rospy.Subscriber("/zhihang2025/first_man/pose", Pose, self._critical_man_cb)
        rospy.Subscriber("/zhihang2025/third_man/pose", Pose, self._healthy_man_cb)

        self.critical_man_reached_pub = rospy.Publisher("/zhihang2025/first_man/reached",
Bool, queue_size=1)
        self.healthy_man_reached_pub = rospy.Publisher("/zhihang2025/third_man/reached",
Bool, queue_size=1)

        rospy.Subscriber("/zhihang2025/iris_bad_man/pose", Pose, self._critical_complete_cb)
        rospy.Subscriber("/zhihang2025/iris_healthy_man/pose", Pose,
self._healthy_complete_cb)

        rospy.wait_for_service("/iris_0/mavros/cmd/arming")
        self.arming_client = rospy.ServiceProxy("/iris_0/mavros/cmd/arming", CommandBool)
        rospy.wait_for_service("/iris_0/mavros/set_mode")

```

```

self.set_mode_client = rospy.ServiceProxy("/iris_0/mavros/set_mode", SetMode)

self.rate = rospy.Rate(20)
rospy.loginfo("XTDrone Controller Initialized. waiting for connection...")

def _extended_state_cb(self, msg):
    self.landed_state = msg.landed_state

def _state_cb(self, msg):
    self.current_state = msg

def _pose_cb(self, msg):
    self.current_pose = msg.pose
    self.position_received = True

    # --- 从四元数中提取偏航角 ---
    orientation_q = self.current_pose.orientation
    _, _, self.current_yaw = tf.transformations.euler_from_quaternion(
        [orientation_q.x, orientation_q.y, orientation_q.z, orientation_q.w])

def _image_callback(self, msg):
    try:
        cv_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        cv2.imshow("Iris Camera", cv_image)
        cv2.waitKey(1)
    except CvBridgeError as e:
        rospy.logerr(e)

def _start_cb(self, msg):
    if msg.data:
        self.can_start = True
        rospy.loginfo("Mission Part1 completed, Part2 start")

def _critical_man_cb(self, msg):
    if self.critical_man_pose is None:
        rospy.loginfo("Received pose for CRITICAL person.")
    self.critical_man_pose = msg

def _healthy_man_cb(self, msg):
    if self.healthy_man_pose is None:
        rospy.loginfo("Received pose for HEALTHY person.")
    self.healthy_man_pose = msg

def _critical_complete_cb(self, msg):
    if msg is not None:
        rospy.loginfo("Searching for CRITICAL person completed.")
        self.critical_complete = True

def _healthy_complete_cb(self, msg):
    if msg is not None:
        rospy.loginfo("Searching for HEALTHY person completed.")
        self.healthy_complete = True

```

```

def _distance(self, x1, y1, x2, y2):
    return math.sqrt(math.pow(x1 - x2, 2) + math.pow(y1 - y2, 2))

def publish_command(self, command):
    """发布一个字符串命令，例如 'ARM', 'AUTO.TAKEOFF'"""
    cmd_msg = String()
    cmd_msg.data = command
    self.cmd_pub.publish(cmd_msg)
    rospy.loginfo(f"Published command: {command}")

def publish_velocity(self, forward=0.0, leftward=0.0, upward=0.0, angular_z=0.0):
    """发布速度指令（机体坐标系：前左上）"""
    twist_msg = Twist()
    twist_msg.linear.x = forward
    twist_msg.linear.y = leftward
    twist_msg.linear.z = upward
    twist_msg.angular.z = angular_z
    self.cmd_vel_pub.publish(twist_msg)

def offboard(self):
    self.publish_command('OFFBOARD')
    rospy.loginfo("Commanding offboard...")
    rospy.sleep(1)

def arm(self):
    self.publish_command('ARM')
    rospy.loginfo("Commanding arm...")
    rospy.sleep(3)

def takeoff(self):
    # 这里的takeoff飞到2m自动悬停
    self.publish_command('AUTO.TAKEOFF')
    rospy.loginfo("Taking off to 2m...")
    rospy.sleep(5)

def change_altitude(self, height):
    # 实现飞行高度的控制
    rospy.loginfo(f"Changing altitude to {height}m...")
    last_request = rospy.Time.now()
    while abs(height - self.current_pose.position.z) > 0.2:
        if rospy.is_shutdown():
            break
        current_time = rospy.Time.now()
        if self.current_state.mode != "OFFBOARD" and (current_time - last_request >
rospy.Duration(5.0)):
            try:
                if self.set_mode_client(custom_mode='OFFBOARD').mode_sent:
                    rospy.loginfo("OFFBOARD mode enabled")
            except rospy.ServiceException as e:
                rospy.logerr("Service call for OFFBOARD failed: %s" % e)
            last_request = current_time

    dz = height - self.current_pose.position.z

```

```

        upward_vel = 1.0 * dz

        self.publish_velocity(upward=upward_vel)
        self.rate.sleep()

    rospy.loginfo(f"Altitude {self.current_pose.position.z}m reached.")
    self.hover()

def move(self, x, y, vel):
    # 以vel大小的速度水平移动到坐标(x,y)
    rospy.loginfo(f"Moving to ({x}, {y}) at {vel} m/s...")
    last_request = rospy.Time.now()
    h = self.current_pose.position.z
    initial_yaw = self.current_yaw

    while self._distance(self.current_pose.position.x, self.current_pose.position.y, x,
y) > 1.0:
        if rospy.is_shutdown():
            break
        current_time = rospy.Time.now()
        if self.current_state.mode != "OFFBOARD" and (current_time - last_request >
rospy.Duration(5.0)):
            try:
                if self.set_mode_client(custom_mode='OFFBOARD').mode_sent:
                    rospy.loginfo("OFFBOARD mode enabled")
            except rospy.ServiceException as e:
                rospy.logerr("Service call for OFFBOARD failed: %s" % e)
            last_request = current_time

        dx = x - self.current_pose.position.x
        dy = y - self.current_pose.position.y
        dz = h - self.current_pose.position.z
        dis = self._distance(self.current_pose.position.x, self.current_pose.position.y,
x, y)

        vel_x = (dx / dis) * vel
        vel_y = (dy / dis) * vel

        d_yaw = initial_yaw - self.current_yaw
        if d_yaw > math.pi:
            d_yaw -= 2 * math.pi
        elif d_yaw < -math.pi:
            d_yaw += 2 * math.pi
        angular_vel_z = 1.0 * d_yaw

        forward_vel = vel_x * math.cos(self.current_yaw) + vel_y *
math.sin(self.current_yaw)
        leftward_vel = -vel_x * math.sin(self.current_yaw) + vel_y *
math.cos(self.current_yaw)

        self.publish_velocity(forward=forward_vel, leftward=leftward_vel, upward=0.0,
angular_z=angular_vel_z)
        self.rate.sleep()

```

```

        rospy.loginfo(f"Position ({self.current_pose.position.x},
{self.current_pose.position.y}, {self.current_pose.position.z}) reached.")
        self.hover()

def moving_to_health(self):
    rospy.loginfo("waiting for HEALTHY person's location...")
    while not rospy.is_shutdown() and self.healthy_man_pose is None:
        rospy.sleep(1)

    target = self.healthy_man_pose.position
    rospy.loginfo("Navigating to HEALTHY person at (%.2f, %.2f)" % (target.x, target.y))
    self.move(target.x, target.y, vel=4)
    con.change_altitude(8.0)

    rospy.loginfo("HEALTHY person reached. Starting to precise landing.")
    self.healthy_man_reached_pub.publish(Bool(True))
    self.hover()

    while not rospy.is_shutdown() and self.healthy_complete == False:
        rospy.sleep(1)
    rospy.loginfo("Searching for HEALTHY person completed... Continue to search critical
man...")
    con.change_altitude(8.0)

def moving_to_critical(self):
    rospy.loginfo("waiting for CRITICAL person's location...")
    while not rospy.is_shutdown() and self.critical_man_pose is None:
        rospy.sleep(1)

    target = self.critical_man_pose.position
    rospy.loginfo("Navigating to CRITICAL person at (%.2f, %.2f)" % (target.x,
target.y))
    self.move(target.x, target.y, vel=4)
    con.change_altitude(4.0)

    rospy.loginfo("CRITICAL person reached. Starting to precise landing.")
    self.critical_man_reached_pub.publish(Bool(True))
    self.hover()

    while not rospy.is_shutdown() and self.critical_complete == False:
        rospy.sleep(1)
    rospy.loginfo("Searching for CRITICAL person completed...")
    con.change_altitude(40)

def hover(self):
    self.publish_command('HOVER')
    self.publish_velocity()

def return_home(self):
    self.publish_command('AUTO.RTL')
    rospy.loginfo("Returning home...")

```

```

def disarm(self):
    rospy.loginfo("waiting for drone to land...")
    while self.landed_state != ExtendedState.LANDED_STATE_ON_GROUND:
        if rospy.is_shutdown():
            break
        rospy.loginfo_throttle(2, "Drone is still in the air...")
        self.rate.sleep()
    rospy.loginfo("Drone has landed.")

    disarm_start_time = rospy.Time.now()
    while self.current_state.armed:
        if rospy.is_shutdown():
            rospy.logwarn("Shutdown requested during disarm process.")
            return

        if rospy.Time.now() - disarm_start_time > rospy.Duration(10.0):
            rospy.logerr("DISARM TIMEOUT! Failed to disarm the vehicle after 30
seconds.")
            return

        # 每隔2秒发送一次解锁指令，确保飞控能收到
        rospy.loginfo_throttle(2, "Vehicle is still armed. Sending DISARM command...")
        self.publish_command('DISARM')

        self.rate.sleep()

    self.publish_command('DISARM')
    rospy.loginfo("Commanding disarm...")

if __name__ == '__main__':
    try:
        con = XTDroneController()
        rospy.loginfo("Quadcopter is waiting for part1 completed...")
        rate = rospy.Rate(1)
        while not rospy.is_shutdown() and not con.can_start:
            rospy.loginfo_throttle(10, "waiting for signal...")
            rate.sleep()
        rospy.loginfo("Signal received! Continuing to part2")

        con.offboard()
        con.arm()
        con.takeoff()

        con.change_altitude(40)
        con.move(1200, -250, 10)
        con.move(1450, -250, 10)
        con.change_altitude(12)

        con.moving_to_health()
        con.moving_to_critical()

        con.move(1450, 250, 10)
        con.move(1200, 250, 10)

```

```

        con.return_home()
        con.disarm()

except rospy.ROSInterruptException:
    pass

```

8.4 detect_red.py 代码及注释

```

# detect_red.py
from collections import deque
from unittest import result
import rospkg
import os
import rospy
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped, Pose, Quaternion, Twist, TwistStamped
from mavros_msgs.msg import State, ExtendedState
from mavros_msgs.srv import CommandBool, SetMode
from std_msgs.msg import String, Bool
from cv_bridge import CvBridge, CvBridgeError
import cv2
from ultralytics import YOLO
import numpy as np
from tf.transformations import euler_from_quaternion, quaternion_matrix
from scipy.spatial.transform import Rotation as R
import threading
from queue import Queue, Empty
import queue
from message_filters import Subscriber, ApproximateTimeSynchronizer
import tf
import math
from simple_pid import PID

class ObjectTrackingHoverNode:
    def __init__(self):
        rospy.init_node('object_tracking_hover_node', anonymous=True)

        # 队列和锁
        self.image_queue = Queue(maxsize=3)
        self.result_queue = Queue()
        self.camera_info = None
        self.drone_pose = None
        self.camera_info_lock = threading.Lock()
        self.drone_pose_lock = threading.Lock()
        self.target_lock = threading.Lock()
        self.vehicle_id = "iris_0"

        # 状态变量
        self.current_state = State()

```

```

self.current_yaw = 0.0
self.start_tracking = False
self.target = None
self.target_found = False
self.critical_pose = Pose()
self.target_pose_published = False

# CV和YOLO
self.bridge = CvBridge()

rospack = rospkg.RosPack()
pkg_path = rospack.get_path('offboard_run')
model_path = os.path.join(pkg_path, 'models', 'red.pt')
self.model = YOLO(model_path)

# 速率
self.image_rate = rospy.Rate(20)
self.control_rate = rospy.Rate(20)
self.rate = rospy.Rate(20)

self.target_3d_queues = {
    "red": deque(maxlen=5),
    "yellow": deque(maxlen=5),
    "white": deque(maxlen=5)
}

self.landing_state = "SEARCHING" # Initial state: SEARCHING, ALIGNING, DESCENDING
self.center_tolerance_px = 45 # 中心区域的容忍度（像素），+/- 20像素
self.center_low = 15 #
self.descend_speed_ms = -0.8 # 下降速度（m/s），负数表示向下
self.final_altitude_m = 0.5 # 最终悬停高度（m）

# Camera parameters (will be updated from CameraInfo)
self.camera_center_x = None
self.camera_center_y = None

self._init_subscribers()
self._init_publishers()
self._init_services()
self._init_threads()
self._init_tf()

rospy.loginfo("Object Tracking Hover Node Initialized.")

def _init_tf(self):
    # 相机朝下安装，定义相机到机体坐标变换矩阵
    self.T_cam2body = np.eye(4)
    self.T_cam2body[:3, :3] = np.array([
        [ 0, -1,  0],
        [-1,  0,  0],
        [ 0,  0, -1]
    ])
    self.T_cam2body[:3, 3] = [0.0, 0.0, -0.03]

```

```

def _init_subscribers(self):
    # 修改话题名称以适配标准PX4+MAVROS设置
    vehicle_id = self.vehicle_id

    # 状态订阅 - 使用标准MAVROS话题
    rospy.Subscriber(f'/{vehicle_id}/mavros/state', State, self._state_cb)
    rospy.Subscriber(f'/{vehicle_id}/mavros/vision_pose/pose', PoseStamped,
self._pose_cb)

    # 图像和相机信息订阅 - 修改为标准Gazebo相机话题
    self.image_sub = Subscriber(f'/{vehicle_id}/camera/image_raw', Image)
    self.camera_info_sub = Subscriber(f'/{vehicle_id}/camera/camera_info', CameraInfo)
    self.pose_sub = Subscriber(f'/{vehicle_id}/mavros/vision_pose/pose', PoseStamped)

    # 同步订阅
    self.sync = ApproximateTimeSynchronizer(
        [self.image_sub, self.camera_info_sub, self.pose_sub],
        queue_size=10,
        slop=0.1
    )
    self.sync.registerCallback(self.synchronized_callback)

    # 开始信号订阅
    rospy.Subscriber('/zhihang2025/first_man/reached', Bool, self._start_tracking_cb)
    rospy.Subscriber('/zhihang2025/first_man/pose', Pose, self._target_pose_cb)

def _init_publishers(self):
    vehicle_id = self.vehicle_id
    # 使用标准MAVROS速度控制话题
    self.velocity_pub =
rospy.Publisher(f'/{vehicle_id}/mavros/setpoint_velocity/cmd_vel', TwistStamped,
queue_size=10)
    rospy.loginfo("Velocity publisher initialized")
    # 备用发布者（如果需要）
    self.cmd_pub = rospy.Publisher(f'/xtdrone/{vehicle_id}/cmd', String, queue_size=1)
    self.mavros_vel_pub = rospy.Publisher(f'/xtdrone/{vehicle_id}/cmd_vel_flu', Twist,
queue_size=10)
    rospy.loginfo("MAVROS velocity publisher initialized")
    # 使用标准消息类型替代自定义消息
    self.object_pose_pub = rospy.Publisher('/detected_object_pose', PoseStamped,
queue_size=10)
    self.pose_publishers = {
        "white": rospy.Publisher('/zhihang2025/iris_healthy_man/pose', Pose,
queue_size=10),
        "red": rospy.Publisher('/zhihang2025/iris_bad_man/pose', Pose, queue_size=10)
    }
    rospy.loginfo("Publishers initialized")

def _init_services(self):
    vehicle_id = self.vehicle_id
    try:
        # 等待MAVROS服务

```

```

        rospy.wait_for_service(f'{vehicle_id}/mavros/cmd/arming', timeout=5.0)
        rospy.wait_for_service(f'{vehicle_id}/mavros/set_mode', timeout=5.0)
        self.arming_client = rospy.ServiceProxy(f'{vehicle_id}/mavros/cmd/arming',
CommandBool)
        self.set_mode_client = rospy.ServiceProxy(f'{vehicle_id}/mavros/set_mode',
SetMode)

        self.use_mavros_services = True
        rospy.loginfo("MAVROS services available")
    except rospy.ROSException:
        rospy.logerr("MAVROS services not available! Please check your PX4+MAVROS
setup")

        self.use_mavros_services = False

def _init_threads(self):
    # 图像处理线程
    self.detection_thread = threading.Thread(target=self._detection_worker)
    self.detection_thread.daemon = True
    self.detection_thread.start()

    # 控制线程
    self.control_thread = threading.Thread(target=self._control_worker)
    self.control_thread.daemon = True
    self.control_thread.start()

def _state_cb(self, msg):
    self.current_state = msg

def _pose_cb(self, msg):
    with self.drone_pose_lock:
        self.drone_pose = msg
        # 提取偏航角
        orientation_q = msg.pose.orientation
        _, _, self.current_yaw = tf.transformations.euler_from_quaternion(
            [orientation_q.x, orientation_q.y, orientation_q.z, orientation_q.w])

def _start_tracking_cb(self, msg):
    if msg.data:
        self.start_tracking = True
        rospy.loginfo("Received start tracking signal - beginning target tracking mode")

def _target_pose_cb(self, msg):
    self.critical_pose = msg

def synchronized_callback(self, image_msg, camera_info_msg, pose_msg):
    """同步回调函数"""
    with self.camera_info_lock:
        if self.camera_info is None: # 只在第一次接收时赋值
            self.camera_info = camera_info_msg
            self.camera_matrix = np.array(camera_info_msg.K).reshape((3, 3))
            self.dist_coeffs = np.array(camera_info_msg.D)
            self.camera_center_x = self.camera_matrix[0, 2]
            self.camera_center_y = self.camera_matrix[1, 2]

```

```

        rospy.loginfo(f"Camera center initialized to ({self.camera_center_x},
{self.camera_center_y})")

    with self.drone_pose_lock:
        self.drone_pose = pose_msg

    if not self.image_queue.full():
        self.image_queue.put((image_msg, pose_msg))
    else:
        try:
            self.image_queue.get_nowait()
            self.image_queue.put((image_msg, pose_msg))
        except:
            pass

def _detection_worker(self):
    """图像处理工作线程"""
    while not rospy.is_shutdown():
        try:
            if not self.start_tracking:
                rospy.sleep(0.1)
                continue

            with self.camera_info_lock:
                if self.camera_info is None:
                    continue

            msg, drone_pose = self.image_queue.get(timeout=1.0)

            cv_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
            results = self.model(source=cv_image, verbose=False)

            self.target_found = False
            best_target = None
            best_conf = 0.0

            for r in results:
                boxes = r.boxes
                for box in boxes:
                    x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
                    conf = float(box.conf.item())
                    cls = int(box.cls.item())

                    if conf > best_conf and conf > 0.7: # 添加置信度阈值
                        best_conf = conf
                        u_pixel = (x1 + x2) / 2
                        v_pixel = (y1 + y2) / 2
                        world_coords = self.get_target_position(u_pixel, v_pixel,
drone_pose.pose)

                        if world_coords is not None:
                            best_target = {

```

```

        'center': (u_pixel, v_pixel),
        'world_coords': world_coords,
        'confidence': conf,
        'bbox': (x1, y1, x2, y2),
        'type': self.model.names[c1s],
        'u_pixel': u_pixel,
        'v_pixel': v_pixel
    }
    self.target_found = True

    # 绘制边界框
    cv2.rectangle(cv_image, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(cv_image, f"{self.model.names[c1s]} {conf:.2f}",
                (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255,
0), 2)

    if best_target:
        self.target = best_target

        # 发布检测到的目标位置（使用标准消息）
        target_pose = PoseStamped()
        target_pose.header.stamp = rospy.Time.now()
        target_pose.header.frame_id = "map"
        target_pose.pose.position.x = best_target['world_coords'][0]
        target_pose.pose.position.y = best_target['world_coords'][1]
        target_pose.pose.position.z = best_target['world_coords'][2]
        self.object_pose_pub.publish(target_pose)

        rospy.loginfo(f"Target found at pixel: {self.target}, world:
{best_target['world_coords']}")
    else:
        self.target = None
        self.target_found = False
        rospy.loginfo("No target found")

    # 显示图像
    scale = 0.5
    resized_image = cv2.resize(cv_image, (0, 0), fx=scale, fy=scale)
    cv2.imshow("Target Tracking", resized_image)
    cv2.waitKey(1)

except queue.Empty:
    continue
except Exception as e:
    rospy.logerr(f"Detection error: {e}")

self.image_rate.sleep()

def _control_worker(self):
    """
    控制工作线程，实现“对准-下降”（Align-and-Descend）的精确降落逻辑。
    """
    rospy.loginfo("Control worker started with Align-and-Descend logic.")

```

```

while not rospy.is_shutdown():

    if not self.start_tracking:
        rospy.sleep(0.1)
        continue

    if self.drone_pose is None or self.camera_info is None:
        #self._publish_velocity_command(0.0, 0.0, 0.0) # 保持悬停
        self.hover()
        self.landing_state = "SEARCHING"
        rospy.sleep(0.1)
        continue

    # ----- 状态机逻辑 -----
    self._ensure_offboard_mode()
    vel = 0.5
    current_altitude = self.drone_pose.pose.position.z
    if current_altitude <= 1.0:
        if not self.target_pose_published and current_altitude <= 0.5:
            self._publish_velocity_command()
            self.hover()
            pose_msg = Pose()
            pose_msg.position.x = self.drone_pose.pose.position.x
            pose_msg.position.y = self.drone_pose.pose.position.y
            pose_msg.position.z = self.target['world_coords'][2]
            self.pose_publishers[self.target['type']].publish(pose_msg)
            rospy.loginfo(f"SUCCESS: Target '{self.target['type']}' pose
{pose_msg.position.x}, {pose_msg.position.y}, {pose_msg.position.z} published.")
            self.target_pose_published = True

        else:
            self._publish_velocity_command(0.0, 0.0, -0.4)
            continue

    # 状态1: 搜索目标
    if self.target is None:
        self.landing_state = "SEARCHING"
        rospy.loginfo_throttle(2, f"State: {self.landing_state} - No target
detected. Hovering.")
        dx = self.critical_pose.position.x - self.drone_pose.pose.position.x
        dy = self.critical_pose.position.y - self.drone_pose.pose.position.y
        dist_to_search = math.sqrt(dx**2 + dy**2)
        if dist_to_search > 2.0:
            rospy.loginfo_throttle(2, f"State: {self.landing_state} - No target.
Moving to search pose. Distance: {dist_to_search:.2f}m")
            # 使用比例控制, 计算世界坐标系下的瞬时速度
            vel_x_world = (dx / dist_to_search) * 1.5
            vel_y_world = (dy / dist_to_search) * 1.5

            # 将世界速度转换为机体速度
            forward_vel = vel_x_world * math.cos(self.current_yaw) + vel_y_world *
math.sin(self.current_yaw)

```

```

        leftward_vel = -vel_x_world * math.sin(self.current_yaw) + vel_y_world *
math.cos(self.current_yaw)
        self._publish_velocity_command(forward_vel, leftward_vel, 0.0)
    else:
        # 到达搜索点附近，悬停等待
        rospy.loginfo_throttle(5, f"State: {self.landing_state} - Arrived at
search pose. Hovering.")
        self._publish_velocity_command()

        self.control_rate.sleep()
        continue

    # 获取当前高度和目标像素位置
    target_u = self.target['u_pixel']
    target_v = self.target['v_pixel']

    # 计算像素误差（目标相对于中心）
    # error > 0 表示目标在中心的右侧/下方
    # error < 0 表示目标在中心的左侧/上方
    error_x_px = target_u - self.camera_center_x
    error_y_px = target_v - self.camera_center_y
    dx = self.target['world_coords'][0] - self.drone_pose.pose.position.x
    dy = self.target['world_coords'][1] - self.drone_pose.pose.position.y
    horizontal_distance = math.sqrt(dx**2 + dy**2)

    # 检查是否在中心容忍度范围内
    if (current_altitude <= 2.0):
        is_aligned = (abs(error_x_px) < self.center_low) and \
            (abs(error_y_px) < self.center_low)
        vel = 0.2
    else:
        is_aligned = (abs(error_x_px) < self.center_tolerance_px) and \
            (abs(error_y_px) < self.center_tolerance_px)

    # 状态2：水平对准
    if not is_aligned:
        self.landing_state = "ALIGNING"
        rospy.loginfo_throttle(1, f"State: {self.landing_state} - Error(px): x=
{error_x_px:.1f}, y={error_y_px:.1f}")

        vel_x_world = (dx / horizontal_distance) * vel
        vel_y_world = (dy / horizontal_distance) * vel

        forward_vel = vel_x_world * math.cos(self.current_yaw) + vel_y_world *
math.sin(self.current_yaw)
        leftward_vel = -vel_x_world * math.sin(self.current_yaw) + vel_y_world *
math.cos(self.current_yaw)

        self._publish_velocity_command(forward_vel, leftward_vel, 0.0)

    # 状态3：垂直下降
    else:
        # 未到达最终高度，继续下降

```

```

        self._publish_velocity_command(0, 0, 0)
        self.landing_state = "DESCENDING"
        rospy.loginfo_throttle(1, f"State: {self.landing_state} - Aligned.
Descending to {self.final_altitude_m}m. Current: {current_altitude:.2f}m")
        self._publish_velocity_command(0.0, 0.0, self.descend_speed_ms)

    self.control_rate.sleep()

def _ensure_offboard_mode(self):
    """确保飞行器处于OFFBOARD模式"""
    if self.use_mavros_services:
        if (self.current_state.mode != "OFFBOARD" and
            hasattr(self, '_last_mode_request') and
            (rospy.Time.now() - self._last_mode_request).to_sec() > 2.0):
            try:
                if self.set_mode_client(custom_mode='OFFBOARD').mode_sent:
                    rospy.loginfo("OFFBOARD mode enabled via MAVROS")
                    self._last_mode_request = rospy.Time.now()
            except rospy.ServiceException as e:
                rospy.logerr(f"MAVROS service call failed: {e}")

        if not hasattr(self, '_last_mode_request'):
            self._last_mode_request = rospy.Time.now()
        self.publish_command('OFFBOARD')

def _distance(self, x1, y1, x2, y2):
    return math.sqrt(math.pow(x1 - x2, 2) + math.pow(y1 - y2, 2))

def publish_command(self, command):
    """发布一个字符串命令, 例如 'ARM', 'AUTO.TAKEOFF'"""
    cmd_msg = String()
    cmd_msg.data = command
    self.cmd_pub.publish(cmd_msg)
    rospy.loginfo(f"Published command: {command}")

def _publish_velocity_command(self, vx=0.0, vy=0.0, vz=0.0, az=0.0):
    """发布速度指令到MAVROS,发布速度指令 (机体坐标系: 前左上)"""
    # 发布Twist消息
    twist = Twist()
    twist.linear.x = vx
    twist.linear.y = vy
    twist.linear.z = vz
    twist.angular.x = 0.0
    twist.angular.y = 0.0
    twist.angular.z = az
    self.mavros_vel_pub.publish(twist)

def move(self, x, y, vel):
    # 以vel大小的速度水平移动到坐标(x,y)
    rospy.loginfo(f"Moving to ({x}, {y}) at {vel} m/s...")
    last_request = rospy.Time.now()
    h = self.drone_pose.pose.position.z
    initial_yaw = self.current_yaw

```

```

        while self._distance(self.drone_pose.pose.position.x,
self.drone_pose.pose.position.y, x, y) > 0.5:
            if rospy.is_shutdown():
                break
            current_time = rospy.Time.now()
            if self.current_state.mode != "OFFBOARD" and (current_time - last_request >
rospy.Duration(5.0)):
                try:
                    if self.set_mode_client(custom_mode='OFFBOARD').mode_sent:
                        rospy.loginfo("OFFBOARD mode enabled")
                    except rospy.ServiceException as e:
                        rospy.logerr("Service call for OFFBOARD failed: %s" % e)
                        last_request = current_time

            dx = x - self.drone_pose.pose.position.x
            dy = y - self.drone_pose.pose.position.y
            dz = h - self.drone_pose.pose.position.z
            dis = self._distance(self.drone_pose.pose.position.x,
self.drone_pose.pose.position.y, x, y)

            vel_x = (dx / dis) * vel
            vel_y = (dy / dis) * vel

            d_yaw = initial_yaw - self.current_yaw
            if d_yaw > math.pi:
                d_yaw -= 2 * math.pi
            elif d_yaw < -math.pi:
                d_yaw += 2 * math.pi
            angular_vel_z = 1.0 * d_yaw

            forward_vel = vel_x * math.cos(self.current_yaw) + vel_y *
math.sin(self.current_yaw)
            leftward_vel = -vel_x * math.sin(self.current_yaw) + vel_y *
math.cos(self.current_yaw)

            self._publish_velocity_command(forward_vel, leftward_vel, 0.0, az=angular_vel_z)
            self.rate.sleep()

            rospy.loginfo(f"Position ({self.drone_pose.pose.position.x},
{self.drone_pose.pose.position.y}, {self.drone_pose.pose.position.z}) reached.")

def hover(self):
    self.publish_command('HOVER')
    self._publish_velocity_command()

# ===== pose calculate =====
def normalize(self, v):
    return v / np.linalg.norm(v)

def pixel_to_camera_ray(self, u, v):
    pixel_h = np.array([u, v, 1.0])
    ray_cam = np.linalg.inv(self.camera_matrix) @ pixel_h

```

```

        return self.normalize(ray_cam)

def camera_ray_to_world(self, ray_cam, orientation):
    # 相机坐标 → 机体坐标
    ray_body = self.T_cam2body[:3, :3] @ ray_cam
    # 机体坐标 → 世界坐标
    q = orientation
    R_world_body = quaternion_matrix([q.x, q.y, q.z, q.w])[:3, :3]
    ray_world = R_world_body @ ray_body
    return self.normalize(ray_world)

def get_target_position(self, u, v, pose, ground_z=0.0):
    if self.camera_info is None or self.drone_pose is None:
        return None

    # 像素坐标转相机方向向量
    ray_cam = self.pixel_to_camera_ray(u, v)

    # 相机方向转世界坐标方向向量
    ray_world = self.camera_ray_to_world(ray_cam, pose.orientation)

    # 计算相机在世界坐标系中的位置
    cam_offset = self.T_cam2body[:3, 3]
    R_world_body = quaternion_matrix([pose.orientation.x,
                                      pose.orientation.y,
                                      pose.orientation.z,
                                      pose.orientation.w])[:3, :3]
    drone_position = np.array([pose.position.x, pose.position.y, pose.position.z])
    #v1 = np.array([2.5, 2.7, 1.3])
    #v2 = np.array([0, 0, 0])
    #drone_position = drone_position + v1 + v2
    cam_position = drone_position + R_world_body @ cam_offset

    # 计算射线与地面的交点
    t = (ground_z - cam_position[2]) / ray_world[2]
    if t < 0:
        return None

    target_world = cam_position + t * ray_world
    return target_world

def run(self):
    rospy.loginfo("Object Tracking Hover Node is running...")
    rospy.loginfo("waiting for /start_tracking signal to begin target tracking...")
    rospy.spin()

if __name__ == '__main__':
    try:
        node = ObjectTrackingHoverNode()
        rate = rospy.Rate(1)
        rospy.loginfo("Object Tracking Hover Node is running...")
        rospy.loginfo("waiting for /first_man reached signal to begin target tracking...")

```

```

        while not rospy.is_shutdown():
            if node.target_pose_published:
                rospy.loginfo("Ending signal received. Shutting down the node.")
                rospy.signal_shutdown("Mission part 1 completed, shutdown signal received")
                break

            rate.sleep()

    except rospy.ROSInterruptException:
        rospy.loginfo("Node interrupted by user")
    except Exception as e:
        rospy.logfatal(f"An unhandled error occurred: {e}", exc_info=True)
    finally:
        rospy.loginfo("Object Tracking Hover node terminated.")
        cv2.destroyAllWindows()
        exit(0)

```

8.5 detect_white.py 代码及注释

```

# detect_white.py
from collections import deque
from unittest import result
import rospkg
import os
import rospy
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped, Pose, Quaternion, Twist, TwistStamped
from mavros_msgs.msg import State, ExtendedState
from mavros_msgs.srv import CommandBool, SetMode
from std_msgs.msg import String, Bool
from cv_bridge import CvBridge, CvBridgeError
import cv2
from ultralytics import YOLO
import numpy as np
from tf.transformations import euler_from_quaternion, quaternion_matrix
from scipy.spatial.transform import Rotation as R
import threading
from queue import Queue, Empty
import queue
from message_filters import Subscriber, ApproximateTimeSynchronizer
import tf
import math
from simple_pid import PID
import copy

class ObjectTrackingHoverNode:
    def __init__(self):
        rospy.init_node('object_tracking_hover_node', anonymous=True)

        # 队列和锁

```

```

self.image_queue = Queue(maxsize=3)
self.result_queue = Queue()
self.camera_info = None
self.drone_pose = None
self.camera_info_lock = threading.Lock()
self.drone_pose_lock = threading.Lock()
self.target_lock = threading.Lock()
self.vehicle_id = "iris_0"

# 状态变量
self.current_state = State()
self.current_yaw = 0.0
self.start_tracking = False
self.target = None
self.target_found = False
self.healthy_pose = Pose()
self.target_pose_published = False
self.current_pos = Pose()

# CV和YOLO
self.bridge = CvBridge()

rospack = rospkg.RosPack()
pkg_path = rospack.get_path('offboard_run')
model_path = os.path.join(pkg_path, 'models', 'stage2.pt')
self.model = YOLO(model_path)

# 速率
self.image_rate = rospy.Rate(20)
self.control_rate = rospy.Rate(20)
self.rate = rospy.Rate(20)

self.landing_state = "SEARCHING" # Initial state: SEARCHING, ALIGNING, DESCENDING

self.motion_data_points = [] # 用于存储目标运动轨迹点 (timestamp, x, y)
self.motion_model = None # 存储计算出的运动模型
self.modeling_min_points = 40 # 开始建模所需的最少数据点
self.intercept_point = None # 存储计算出的拦截点

self.descend_speed_ms = -0.4 # 下降速度 (m/s)
self.final_altitude_m = 0.7 # 最终触发任务完成的高度 (m)
self.sync_altitude_m = 10.0 # 开始同步下降的高度
self.landing_commit_alt = 1.5 # 切换到最终降落阶段的高度
self.prediction_time_s = 0.2 # 预测未来多少秒的位置，用于补偿延迟
self.final_align_tolerance_px = 50 # 中心对准的像素容忍度

# Camera parameters (will be updated from CameraInfo)
self.camera_center_x = None
self.camera_center_y = None

self._init_subscribers()
self._init_publishers()
self._init_services()

```

```

self._init_threads()
self._init_tf()

rospy.loginfo("Object Tracking Hover Node Initialized.")

def _init_tf(self):
    # 相机朝下安装, 定义相机到机体坐标变换矩阵
    self.T_cam2body = np.eye(4)
    self.T_cam2body[:3, :3] = np.array([
        [ 0, -1,  0],
        [-1,  0,  0],
        [ 0,  0, -1]
    ])
    self.T_cam2body[:3, 3] = [0.0, 0.0, -0.03]

def _init_subscribers(self):
    # 修改话题名称以适配标准PX4+MAVROS设置
    vehicle_id = self.vehicle_id # 或者根据你的设置修改

    # 状态订阅 - 使用标准MAVROS话题
    rospy.Subscriber(f'/{vehicle_id}/mavros/state', State, self._state_cb)
    rospy.Subscriber(f'/{vehicle_id}/mavros/local_position/pose', PoseStamped,
self._pose_cb)
    rospy.Subscriber(f'/{vehicle_id}/mavros/vision_pose/pose', PoseStamped,
self._position_cb)

    # 图像和相机信息订阅 - 修改为标准Gazebo相机话题
    self.image_sub = Subscriber(f'/{vehicle_id}/camera/image_raw', Image)
    self.camera_info_sub = Subscriber(f'/{vehicle_id}/camera/camera_info', CameraInfo)
    self.pose_sub = Subscriber(f'/{vehicle_id}/mavros/vision_pose/pose', PoseStamped)

    # 同步订阅
    self.sync = ApproximateTimeSynchronizer(
        [self.image_sub, self.camera_info_sub, self.pose_sub],
        queue_size=3,
        slop=0.1
    )
    self.sync.registerCallback(self.synchronized_callback)

    # 开始信号订阅
    rospy.Subscriber('/zhihang2025/third_man/reached', Bool, self._start_tracking_cb)
    rospy.Subscriber('/zhihang2025/third_man/pose', Pose, self._target_pose_cb)

def _init_publishers(self):
    vehicle_id = self.vehicle_id

    # 发布者
    self.cmd_pub = rospy.Publisher(f'/xtdrone/{vehicle_id}/cmd', String, queue_size=1)
    self.mavros_vel_pub = rospy.Publisher(f'/xtdrone/{vehicle_id}/cmd_vel_flu', Twist,
queue_size=10)
    rospy.loginfo("MAVROS velocity publisher initialized")

    # 使用标准消息类型替代自定义消息

```

```

        self.object_pose_pub = rospy.Publisher('/detected_object_pose', PoseStamped,
queue_size=10)
        self.pose_publishers = {
            "white": rospy.Publisher('/zhihang2025/iris_healthy_man/pose', Pose,
queue_size=10),
            "red": rospy.Publisher('/zhihang2025/iris_bad_man/pose', Pose, queue_size=10)
        }
        rospy.loginfo("Publishers initialized")

def _init_services(self):
    vehicle_id = self.vehicle_id

    try:
        # 等待MAVROS服务
        rospy.wait_for_service(f'{vehicle_id}/mavros/cmd/arming', timeout=5.0)
        rospy.wait_for_service(f'{vehicle_id}/mavros/set_mode', timeout=5.0)
        self.arming_client = rospy.ServiceProxy(f'{vehicle_id}/mavros/cmd/arming',
CommandBool)
        self.set_mode_client = rospy.ServiceProxy(f'{vehicle_id}/mavros/set_mode',
SetMode)
        self.use_mavros_services = True
        rospy.loginfo("MAVROS services available")
    except rospy.ROSException:
        rospy.logerr("MAVROS services not available! Please check your PX4+MAVROS
setup")
        self.use_mavros_services = False

def _init_threads(self):
    # 图像处理线程
    self.detection_thread = threading.Thread(target=self._detection_worker)
    self.detection_thread.daemon = True
    self.detection_thread.start()

    # 控制线程
    self.control_thread = threading.Thread(target=self._control_worker)
    self.control_thread.daemon = True
    self.control_thread.start()

def _state_cb(self, msg):
    self.current_state = msg

def _pose_cb(self, msg):
    with self.drone_pose_lock:
        self.drone_pose = msg
        # 提取偏航角
        orientation_q = msg.pose.orientation
        _, _, self.current_yaw = tf.transformations.euler_from_quaternion(
            [orientation_q.x, orientation_q.y, orientation_q.z, orientation_q.w])

def _start_tracking_cb(self, msg):
    if msg.data:
        self.start_tracking = True
        rospy.loginfo("Received start tracking signal - beginning target tracking mode")

```

```

def _target_pose_cb(self, msg):
    self.healthy_pose = msg

def _position_cb(self, msg):
    self.current_pos = msg.pose

def synchronized_callback(self, image_msg, camera_info_msg, pose_msg):
    """同步回调函数"""
    with self.camera_info_lock:
        if self.camera_info is None: # 只在第一次接收时赋值
            self.camera_info = camera_info_msg
            self.camera_matrix = np.array(camera_info_msg.K).reshape((3, 3))
            self.dist_coeffs = np.array(camera_info_msg.D)
            # ADDED: Get camera center from camera info
            self.camera_center_x = self.camera_matrix[0, 2]
            self.camera_center_y = self.camera_matrix[1, 2]
            rospy.loginfo(f"Camera center initialized to ({self.camera_center_x},
{self.camera_center_y})")

    with self.drone_pose_lock:
        self.drone_pose = pose_msg

    if not self.image_queue.full():
        self.image_queue.put((image_msg, pose_msg))
    else:
        try:
            self.image_queue.get_nowait()
            self.image_queue.put((image_msg, pose_msg))
        except Exception as e:
            rospy.logwarn(f"Queue put error: {e}")

def _detection_worker(self):
    """图像处理工作线程"""
    while not rospy.is_shutdown():
        try:
            if not self.start_tracking:
                rospy.sleep(0.1)
                continue

            with self.camera_info_lock:
                if self.camera_info is None:
                    continue

            msg, cam_pose = self.image_queue.get(timeout=1.0)

            cv_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
            results = self.model(source=cv_image, verbose=False)

            self.target_found = False
            best_target = None
            best_conf = 0.0

```

```

for r in results:
    boxes = r.bboxes
    for box in boxes:
        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
        conf = float(box.conf.item())
        cls = int(box.cls.item())

        if conf > best_conf and conf > 0.7: # 添加置信度阈值
            best_conf = conf
            u_pixel = (x1 + x2) / 2
            v_pixel = (y1 + y2) / 2
            world_coords = self.get_target_position(u_pixel, v_pixel,
cam_pose.pose)

            if world_coords is not None:
                best_target = {
                    'center': (u_pixel, v_pixel),
                    'world_coords': world_coords,
                    'confidence': conf,
                    'bbox': (x1, y1, x2, y2),
                    'type': self.model.names[cls],
                    'u_pixel': u_pixel,
                    'v_pixel': v_pixel
                }
                self.target_found = True

            # 绘制边界框
            cv2.rectangle(cv_image, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(cv_image, f"{self.model.names[cls]} {conf:.2f}",
                        (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255,
0), 2)

        if best_target:
            self.target = best_target

            # 发布检测到的目标位置（使用标准消息）
            target_pose = PoseStamped()
            target_pose.header.stamp = rospy.Time.now()
            target_pose.header.frame_id = "map"
            target_pose.pose.position.x = best_target['world_coords'][0]
            target_pose.pose.position.y = best_target['world_coords'][1]
            target_pose.pose.position.z = best_target['world_coords'][2]
            self.object_pose_pub.publish(target_pose)

            rospy.loginfo(f"Target found at pixel: {self.target}, world:
{best_target['world_coords']}")
        else:
            self.target = None
            self.target_found = False
            rospy.loginfo("No target found")

            # 显示图像
            scale = 0.5

```

```

        resized_image = cv2.resize(cv_image, (0, 0), fx=scale, fy=scale)
        cv2.imshow("Target Tracking", resized_image)
        cv2.waitKey(1)

    except queue.Empty:
        continue
    except Exception as e:
        rospy.logerr(f"Detection error: {e}")

    self.image_rate.sleep()

def _control_worker(self):
    """
    控制工作线程，实现“对准-下降”的精确降落逻辑。
    """
    rospy.loginfo("Control worker started with Intercept-and-wait logic.")

    while not rospy.is_shutdown():

        if not self.start_tracking:
            rospy.sleep(0.1)
            continue

        if self.drone_pose is None or self.camera_info is None:
            self.hover()
            self.landing_state = "SEARCHING"
            rospy.sleep(0.1)
            continue

        # ----- 状态机逻辑 -----
        self._ensure_offboard_mode()
        current_altitude = self.drone_pose.pose.position.z
        vel = 1.0

        # 阶段 1: 搜索
        if self.landing_state == "SEARCHING":
            if self.target is None:
                rospy.loginfo_throttle(2, "State: SEARCHING - No target. Moving to initial search pose.")
                dx = self.healthy_pose.position.x - self.drone_pose.pose.position.x
                dy = self.healthy_pose.position.y - self.drone_pose.pose.position.y
                dist = math.sqrt(dx**2 + dy**2)
                if dist > 0.5:
                    rospy.loginfo_throttle(2, f"State: SEARCHING - Moving to search pose, distance: {dist:.2f}m")
                    vel_x = (dx / dist) * vel
                    vel_y = (dy / dist) * vel
                    f_vel = vel_x * math.cos(self.current_yaw) + vel_y * math.sin(self.current_yaw)
                    l_vel = -vel_x * math.sin(self.current_yaw) + vel_y * math.cos(self.current_yaw)
                    self._publish_velocity_command(f_vel, l_vel, 0.0)
                else:

```

```

        rospy.loginfo_throttle(5, "State: SEARCHING - Arrived at search
pose. Hovering.")

        self.hover()
    else:
        rospy.loginfo("Target acquired. Switching to MODELING.")
        self.landing_state = "MODELING"
        self.motion_data_points = [] # 清空旧数据
        self.hover()

# 阶段 2: 运动建模
elif self.landing_state == "MODELING":
    if self.target is None:
        rospy.logwarn("Target lost during modeling. Returning to SEARCHING.")
        self.landing_state = "SEARCHING"
        continue

    # 收集数据点
    current_time = rospy.Time.now().to_sec()
    world_coords = self.target['world_coords']
    self.motion_data_points.append((current_time, world_coords[0],
world_coords[1]))
    rospy.loginfo_throttle(1, f"State: MODELING - Collecting data points
({len(self.motion_data_points)}/{self.modeling_min_points}).")

    # 悬停在同步高度, 以获得稳定的观测视角
    self.hover()

    # 数据足够时, 尝试计算模型
    if len(self.motion_data_points) >= self.modeling_min_points:
        if self._calculate_motion_model():
            rospy.loginfo("Motion model created. Switching to SYNCHRONIZING.")
            self.landing_state = "INTERCEPTING"
            self.intercept_point = None

elif self.landing_state == "INTERCEPTING":
    if self.motion_model is None:
        rospy.logerr("In INTERCEPTING state but no motion model available.
Returning to SEARCHING.")
        self.landing_state = "SEARCHING"
        continue

    # 只计算一次拦截点
    if self.intercept_point is None:

        center = self.motion_model['center']
        radius = self.motion_model['radius']

        self.intercept_point = np.array([center[0], center[1] - radius])
        rospy.loginfo(f"Calculated intercept point:
({self.intercept_point[0]:.2f}, {self.intercept_point[1]:.2f})")

    dx = self.intercept_point[0] - self.drone_pose.pose.position.x
    dy = self.intercept_point[1] - self.drone_pose.pose.position.y

```

```

        dist = math.sqrt(dx**2 + dy**2)
        if dist > 0.5:
            rospy.loginfo_throttle(1, f"State: INTERCEPTING - Moving to intercept
point, distance: {dist:.2f}m")
            vel_x = (dx / dist) * vel
            vel_y = (dy / dist) * vel
            f_vel = vel_x * math.cos(self.current_yaw) + vel_y *
math.sin(self.current_yaw)
            l_vel = -vel_x * math.sin(self.current_yaw) + vel_y *
math.cos(self.current_yaw)
            self._publish_velocity_command(f_vel, l_vel, 0.0)
        else:
            rospy.loginfo("Reached intercept point. Switching to
DESCENDING_TO_WAIT.")
            self.landing_state = "DESCENDING_TO_WAIT"
            self.hover()

# 阶段 4: 下降至等待高度
elif self.landing_state == "DESCENDING_TO_WAIT":
    rospy.loginfo_throttle(1, f"State: DESCENDING_TO_WAIT - Descending to
{self.final_altitude_m}m. Current: {current_altitude:.2f}m")
    # 保持水平位置不动
    self._publish_velocity_command(vx=0.0, vy=0.0, vz=-1.0)

    # 检查是否到达
    if current_altitude <= self.final_altitude_m:
        rospy.loginfo(f"Reached waiting altitude. Switching to
WAITING_FOR_TARGET.")
        self.landing_state = "WAITING_FOR_TARGET"
        self.hover()

# 阶段 5: 等待目标通过
elif self.landing_state == "WAITING_FOR_TARGET":
    rospy.loginfo_throttle(2, "State: WAITING_FOR_TARGET - Hovering at low
altitude, waiting for target.")
    self.hover()
    if self.target is not None:
        pose_msg = Pose()
        pose_msg.position.x = self.intercept_point[0] - 1.23
        pose_msg.position.y = self.intercept_point[1] + 1
        pose_msg.position.z = 0.0
        self.pose_publishers['white'].publish(pose_msg)
        rospy.loginfo(f"SUCCESS: Target ['white'] pose
({self.intercept_point[0], self.intercept_point[1]})published.")
        self.target_pose_published = True # 触发主线程退出
        ...

        error_x_px = self.target['u_pixel'] - self.camera_center_x
        error_y_px = self.target['v_pixel'] - self.camera_center_y
        if (abs(error_x_px) < self.final_align_tolerance_px and
            abs(error_y_px) < self.final_align_tolerance_px):
            pose_msg = Pose()
            pose_msg.position.x = self.intercept_point[0]
            pose_msg.position.y = self.intercept_point[1]

```

```

        pose_msg.position.z = 0.0
        self.pose_publishers['white'].publish(pose_msg)
        rospy.loginfo(f"SUCCESS: Target ['white'] pose
({self.intercept_point[0], self.intercept_point[1]})published.")
        self.target_pose_published = True # 触发主线程退出
    else:
        radius_direction_vec = self.intercept_point -
self.motion_model['center']
        radius_direction_unit_vec = radius_direction_vec /
np.linalg.norm(radius_direction_vec)
        target = self.intercept_point + radius_direction_unit_vec *
self.motion_model['radius']

        dx = target[0] - self.drone_pose.pose.position.x
        dy = target[1] - self.drone_pose.pose.position.y
        dist = math.sqrt(dx**2 + dy**2)

        rospy.loginfo_throttle(1, f"State: ADJUSTING - Moving to final
observation point, distance: {dist:.2f}m")
        vel_x = (dx / dist) * 0.1
        vel_y = (dy / dist) * 0.1
        f_vel = vel_x * math.cos(self.current_yaw) + vel_y *
math.sin(self.current_yaw)
        l_vel = -vel_x * math.sin(self.current_yaw) + vel_y *
math.cos(self.current_yaw)
        self._publish_velocity_command(f_vel, l_vel, 0.0)
        rospy.logwarn("Finalizing but target is lost. Cannot publish.")'''

    ...

    rospy.loginfo(f"{target[0]}, {target[1]}, {self.intercept_point[0]},
{self.intercept_point[1]}")
    if dist > 0.1:
        rospy.loginfo_throttle(1, f"State: ADJUSTING - Moving to final
observation point, distance: {dist:.2f}m")
        vel_x = (dx / dist) * 0.1
        vel_y = (dy / dist) * 0.1
        f_vel = vel_x * math.cos(self.current_yaw) + vel_y *
math.sin(self.current_yaw)
        l_vel = -vel_x * math.sin(self.current_yaw) + vel_y *
math.cos(self.current_yaw)
        self._publish_velocity_command(f_vel, l_vel, 0.0)
    else:
        rospy.loginfo("Reached final observation point. Switching to
OBSERVING.")

        self.hover()
        if self.target is not None:
            pose_msg = Pose()
            pose_msg.position.x = self.current_pos.position.x
            pose_msg.position.y = self.current_pos.position.y
            pose_msg.position.z = 0.0
            self.pose_publishers['white'].publish(pose_msg)

```

```

        rospy.loginfo(f"SUCCESS: Target ['white'] pose
({self.intercept_point[0], self.intercept_point[1]})published.")
        self.target_pose_published = True'''

'''# 检查目标是否在摄像头中心
if self.target is not None:
    error_x_px = self.target['u_pixel'] - self.camera_center_x
    error_y_px = self.target['v_pixel'] - self.camera_center_y
    if (abs(error_x_px) < self.final_align_tolerance_px and
        abs(error_y_px) < self.final_align_tolerance_px):

        rospy.loginfo("Target is in camera center! Publishing the
position.")

        self.landing_state = "FINALIZING"
        pose_msg = Pose()
        pose_msg.position.x = self.current_pos.position.x
        pose_msg.position.y = self.current_pos.position.y
        pose_msg.position.z = 0.0
        self.pose_publishers['white'].publish(pose_msg)
        rospy.loginfo(f"SUCCESS: Target ['white'] pose
({self.intercept_point[0], self.intercept_point[1]})published.")
        self.target_pose_published = True
'''

# 阶段 6: 任务完成
'''elif self.landing_state == "FINALIZING":
    rospy.loginfo("State: FINALIZING - Publishing success topic.")

# 发布任务完成话题
if self.target is not None:
    pose_msg = Pose()
    pose_msg.position.x = self.current_pos.position.x
    pose_msg.position.y = self.current_pos.position.y
    pose_msg.position.z = 0.0
    self.pose_publishers['white'].publish(pose_msg)
    rospy.loginfo(f"SUCCESS: Target ['white'] pose
({self.intercept_point[0], self.intercept_point[1]})published.")
    self.target_pose_published = True # 触发主线程退出
else:
    rospy.logwarn("Finalizing but target is lost or not white. Cannot
publish.")
'''

self.control_rate.sleep()

def _ensure_offboard_mode(self):
    """确保飞行器处于OFFBOARD模式"""
    self.publish_command('OFFBOARD')

def _distance(self, x1, y1, x2, y2):
    return math.sqrt(math.pow(x1 - x2, 2) + math.pow(y1 - y2, 2))

def publish_command(self, command):

```

```

        """发布一个字符串命令，例如 'ARM', 'AUTO.TAKEOFF'"""
        cmd_msg = String()
        cmd_msg.data = command
        self.cmd_pub.publish(cmd_msg)
        rospy.loginfo(f"Published command: {command}")

def _publish_velocity_command(self, vx=0.0, vy=0.0, vz=0.0, az=0.0):
    """发布速度指令到MAVROS,发布速度指令 (机体坐标系: 前左上)"""
    twist = Twist()
    twist.linear.x = vx
    twist.linear.y = vy
    twist.linear.z = vz
    twist.angular.x = 0.0
    twist.angular.y = 0.0
    twist.angular.z = az
    self.mavros_vel_pub.publish(twist)

def move(self, x, y, vel):
    # 以vel大小的速度水平移动到坐标(x,y)
    rospy.loginfo(f"Moving to ({x}, {y}) at {vel} m/s...")
    last_request = rospy.Time.now()
    h = self.drone_pose.pose.position.z
    initial_yaw = self.current_yaw

    while self._distance(self.drone_pose.pose.position.x,
self.drone_pose.pose.position.y, x, y) > 0.5:
        if rospy.is_shutdown():
            break
        current_time = rospy.Time.now()
        if self.current_state.mode != "OFFBOARD" and (current_time - last_request >
rospy.Duration(5.0)):
            try:
                if self.set_mode_client(custom_mode='OFFBOARD').mode_sent:
                    rospy.loginfo("OFFBOARD mode enabled")
            except rospy.ServiceException as e:
                rospy.logerr("Service call for OFFBOARD failed: %s" % e)
            last_request = current_time

        dx = x - self.drone_pose.pose.position.x
        dy = y - self.drone_pose.pose.position.y
        dz = h - self.drone_pose.pose.position.z
        dis = self._distance(self.drone_pose.pose.position.x,
self.drone_pose.pose.position.y, x, y)

        vel_x = (dx / dis) * vel
        vel_y = (dy / dis) * vel

        d_yaw = initial_yaw - self.current_yaw
        if d_yaw > math.pi:
            d_yaw -= 2 * math.pi
        elif d_yaw < -math.pi:
            d_yaw += 2 * math.pi
        angular_vel_z = 1.0 * d_yaw

```

```

        forward_vel = vel_x * math.cos(self.current_yaw) + vel_y *
math.sin(self.current_yaw)
        leftward_vel = -vel_x * math.sin(self.current_yaw) + vel_y *
math.cos(self.current_yaw)

        self._publish_velocity_command(forward_vel, leftward_vel, 0.0, az=angular_vel_z)
        self.rate.sleep()

    rospy.loginfo(f"Position ({self.drone_pose.pose.position.x},
{self.drone_pose.pose.position.y}, {self.drone_pose.pose.position.z}) reached.")

def hover(self):
    self.publish_command('HOVER')
    self._publish_velocity_command()

# ===== pose calculate =====
def normalize(self, v):
    return v / np.linalg.norm(v)

def pixel_to_camera_ray(self, u, v):
    pixel_h = np.array([u, v, 1.0])
    ray_cam = np.linalg.inv(self.camera_matrix) @ pixel_h
    return self.normalize(ray_cam)

def camera_ray_to_world(self, ray_cam, orientation):
    # 相机坐标 → 机体坐标
    ray_body = self.T_cam2body[:3, :3] @ ray_cam
    # 机体坐标 → 世界坐标
    q = orientation
    R_world_body = quaternion_matrix([q.x, q.y, q.z, q.w])[:3, :3]
    ray_world = R_world_body @ ray_body
    return self.normalize(ray_world)

def get_target_position(self, u, v, pose, ground_z=0.0):
    if self.camera_info is None or self.drone_pose is None:
        return None

    # 像素坐标转相机方向向量
    ray_cam = self.pixel_to_camera_ray(u, v)

    # 相机方向转世界坐标方向向量
    ray_world = self.camera_ray_to_world(ray_cam, pose.orientation)

    # 计算相机在世界坐标系中的位置
    cam_position = np.array([pose.position.x, pose.position.y, pose.position.z])

    # 计算射线与地面的交点
    t = (ground_z - cam_position[2]) / ray_world[2]
    if t < 0:
        return None

    target_world = cam_position + t * ray_world

```

```

        return target_world

def _calculate_motion_model(self):
    """
    使用最小二乘法拟合圆形轨迹，并计算角速度。
    """
    if len(self.motion_data_points) < self.modeling_min_points:
        return False

    # 提取x, y坐标
    points = np.array([p[1:] for p in self.motion_data_points])
    x = points[:, 0]
    y = points[:, 1]

    # 最小二乘法拟合圆
    #  $x^2 + y^2 + a*x + b*y + c = 0$ 
    A = np.vstack([x, y, np.ones(len(x))]).T
    B = -(x**2 + y**2)

    try:
        # 求解 a, b, c
        coeffs, _, _, _ = np.linalg.lstsq(A, B, rcond=None)
        a, b, c = coeffs

        # 计算圆心和半径
        cx = -a / 2
        cy = -b / 2
        radius = np.sqrt(cx**2 + cy**2 - c)
    except np.linalg.LinAlgError:
        rospy.logwarn("Circle fitting failed. Not enough unique points.")
        self.motion_data_points = [] # 清空数据重新收集
        return False

    # 计算角速度
    angles = [math.atan2(p[2] - cy, p[1] - cx) for p in self.motion_data_points]
    times = [p[0] for p in self.motion_data_points]

    # 使用差分计算角速度，并取平均值
    angular_velocities = []
    for i in range(1, len(angles)):
        delta_angle = angles[i] - angles[i-1]
        # 处理角度跳跃 (从 +pi 到 -pi 或反之)
        if delta_angle > math.pi:
            delta_angle -= 2 * math.pi
        elif delta_angle < -math.pi:
            delta_angle += 2 * math.pi

        delta_time = times[i] - times[i-1]
        if delta_time > 1e-6: # 避免除以零
            angular_velocities.append(delta_angle / delta_time)

    if not angular_velocities:

```

```

        rospy.logwarn("Could not calculate angular velocity.")
        return False

    omega = np.mean(angular_velocities)

    # 半径必须在合理范围内
    if not (2.93 < radius < 3.07) or not (1493.07 < cx < 1494.47):
        rospy.logwarn(f"Calculated radius {radius:.2f}m is out of expected range.
Retrying.")
        rospy.logwarn(f"Calculated center_x {cx:.2f}m is out of expected range.
Retrying.")
        self.motion_data_points = [] # 清空数据重新收集
        return False

    # 保存模型
    self.motion_model = {
        'center': np.array([cx, cy]),
        'radius': radius,
        'omega': omega,
        'last_angle': angles[-1],
        'last_time': times[-1]
    }
    rospy.loginfo(f"Motion model UPDATED: Center({cx:.2f}, {cy:.2f}), Radius=
{radius:.2f}, Omega={omega:.2f} rad/s")
    return True

def _predict_target_position(self):
    """
    根据已建立的运动模型，预测目标在未来某个时刻的位置。
    """
    if self.motion_model is None:
        return None

    model = self.motion_model

    # 计算从上次更新模型到现在的时间差
    time_since_last_update = rospy.Time.now().to_sec() - model['last_time']

    # 加上我们想要预测的未来时间
    total_time_delta = time_since_last_update + self.prediction_time_s

    # 预测新的角度
    predicted_angle = model['last_angle'] + model['omega'] * total_time_delta

    # 计算预测的世界坐标
    cx, cy = model['center']
    R = model['radius']
    predicted_x = cx + R * math.cos(predicted_angle)
    predicted_y = cy + R * math.sin(predicted_angle)

    return np.array([predicted_x, predicted_y])

def run(self):

```

```

    rospy.loginfo("Object Tracking Hover Node is running...")
    rospy.loginfo("Waiting for /start_tracking signal to begin target tracking...")
    rospy.spin()

if __name__ == '__main__':
    try:
        node = ObjectTrackingHoverNode()
        rate = rospy.Rate(1)
        rospy.loginfo("Object Tracking Hover Node is running...")
        rospy.loginfo("Waiting for /start_tracking signal to begin target tracking...")
        while not rospy.is_shutdown():
            if node.target_pose_published:
                rospy.loginfo("Ending signal received. Shutting down the node.")
                rospy.signal_shutdown("Mission part 1 completed, shutdown signal received")
                break

            rate.sleep()
    except rospy.ROSInterruptException:
        rospy.loginfo("Node interrupted by user")
    except Exception as e:
        rospy.logfatal(f"An unhandled error occurred: {e}", exc_info=True)
    finally:
        rospy.loginfo("Object Tracking Hover node terminated.")
        cv2.destroyAllWindows()
        exit(0)

```

9. 运行须知

9.1 启动方式

要启动该项目，请运行与当前文件同级目录下的 `launch.sh` 脚本，该脚本将启动所有必要的节点和服务，确保无人机能够正确执行路径点导航和目标识别任务。

```

# 启动地面站
cd ~/Downloads # 你安装的地面站位置
./QGroundControl.AppImage
# 激活启动脚本
cd ~/catkin_ws/src/offboard_run/scripts
./launch.sh

```

此处启动时若遇到问题详见 [使用说明.pdf](#)

9.2 依赖项

确保已安装以下依赖项：

1. OpenCV

版本建议'4.10.0'

可以通过以下命令安装 OpenCV：

```
sudo apt-get install python3-opencv
```

2. YOLOv8

您需要安装 ultralytics 包来使用 YOLOv8 模型：

```
pip install ultralytics
```

3. cv_bridge

请确保已安装 cv_bridge 以便将 ROS 图像消息转换为 OpenCV 格式：

```
sudo apt-get install ros--cv-bridge
```

4. gazebo_msgs

请确保已安装 gazebo_msgs 以便与 Gazebo 进行通信：

```
sudo apt-get install ros--gazebo-msgs
```